

VeriGate: An Adaptive, Self-Verifying Semantic Cache for Scalable and Cost-Efficient LLM Gateways

A report submitted for the course named Project - II (CS4201)

Submitted By

ABHISHEK YADAV
SEMESTER - VII
230103041

Supervised By

DR. SALAM MICHAEL SINGH



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR
NOVEMBER, 2026

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

ABHISHEK YADAV
230103041



Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. Salam Michael Singh
Assistant Professor

Email: michael_s@iiitmanipur.ac.in
Contact No: +91 8575443081

To Whom It May Concern

This is to certify that the project report entitled "**VeriGate: An Adaptive, Self-Verifying Semantic Cache for Scalable and Cost-Efficient LLM Gateways**", submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fulfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of work carried out by **ABHISHEK YADAV** bearing roll number 230103041.

Signature of Supervisor

(Dr. Salam Michael Singh)

Signature of the Examiner 1

Signature of the Examiner 2

Signature of the Examiner 3

Signature of the Examiner 4

Abstract

Large Language Models (LLMs) are increasingly deployed in production, but every API call incurs monetary cost and high latency. A *semantic cache* mitigates this by returning a stored answer when a new query is semantically “close enough” to a past one. However, existing semantic caches (e.g. GPTCache) rely on a **single fixed similarity threshold**, which produces **false cache hits** — serving the wrong stored answer to a look-alike query (for example, returning the answer for “*capital of Austria*” to “*capital of Australia*”). This report presents **VeriGate**, a production-grade LLM Gateway whose core novel contribution is an **adaptive, self-verifying semantic cache**: it (i) adapts the similarity threshold per query using neighbourhood density, (ii) *verifies* a candidate hit with a two-tier verifier (a near-free structural check on numbers, negation and named entities, and an optional NLI cross-encoder), and (iii) detects volatile queries to skip caching time-sensitive answers. A second contribution, a **cost-aware model cascade**, routes easy queries to a cheap model and escalates only hard ones. On a controlled benchmark, VeriGate reduces the false-hit rate from the fixed-threshold baseline’s **46.7%** to **0.0%** (100% precision), with structural verification costing only **~0.03 ms**; a vectorised index brings the cache-hit path to **~10 ms** ($\sim 78\times$ faster than an 800 ms LLM call), and the cascade cuts LLM cost by **~47%** with no quality loss on hard queries. The system is engineered for production with per-tenant cache isolation, per-key spend budgets, a provider circuit breaker, Prometheus/Grafana observability, and Docker packaging.

Acknowledgement

I would like to express my deepest appreciation to all those who provided me the possibility to complete this project. A special gratitude I give to my supervisor, **Dr. Salam Michael Singh**, whose contribution in stimulating suggestions and encouragement helped me to coordinate my project especially in writing this report.

I would also like to express my sincere gratitude to the Head of the Department (CSE), **Dr. Navanth Saharia**, for his constant support and for providing the necessary facilities to successfully execute this project.

Furthermore, I would also like to acknowledge with much appreciation the crucial role of the faculty and staff of the Department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur, who gave the permission to use all required equipment and necessary materials to complete the tasks.

Last but not least, many thanks go to my family and friends whose support, guidance, and encouragement have been invaluable throughout this endeavor.

ABHISHEK YADAV

Roll No: 230103041

Contents

Abstract	3
Acknowledgement	4
List of Figures	8
List of Tables	10
1 Introduction	11
1.1 Background	11
1.2 Motivation	11
1.3 Problem Definition	12
1.4 Project Objectives	13
1.5 Scope of the Project	13
1.6 Project Timeline	13
2 Literature Survey	15
2.1 Overview	15
2.2 Large Language Models and Efficient Inference	15
2.3 Semantic Caching for LLM Applications	15
2.4 Text Embeddings and Semantic Similarity	16
2.5 Approximate Nearest Neighbour Search and Vector Indexing	16
2.6 Natural Language Inference and Answer Verification	16
2.7 Cost-Aware Routing and Model Cascades	17
2.8 Gap Analysis and VeriGate’s Contribution	17
3 Requirement Engineering	18
3.1 Introduction	18
3.2 System Analysis	18
3.2.1 Limitations of the Existing System	18
3.2.2 Feasibility Study	19
3.3 System Requirements	19

3.3.1	Hardware Requirements	19
3.3.2	Software Requirements	19
3.4	Functional Requirements	20
3.5	Non-Functional Requirements	21
4	System Design	22
4.1	Introduction to System Design	22
4.2	Design Goals and Guiding Principles	22
4.3	Design Patterns Employed	23
4.4	System Architecture	23
4.5	Use Case Analysis	24
4.6	Data Flow Diagrams	25
4.6.1	Level-0 DFD (Context Diagram)	25
4.6.2	Level-1 DFD (Process Decomposition)	26
4.7	Request Lifecycle and the Cache Decision	26
4.7.1	Request Sequence (HIT vs MISS)	28
4.8	Cache Intelligence Design	28
4.8.1	Adaptive Similarity Threshold	28
4.8.2	Two-Tier Self-Verifier	28
4.8.3	Volatility Detection	29
4.9	Cost-Aware Cascade Design	29
4.10	Component and Module Design	29
4.11	Data Model and Tenant Namespacing	30
4.12	API Route Design	31
4.13	Deployment Topology	31
4.14	Failure-Mode and Error-Handling Design	32
5	Implementation	34
5.1	Introduction	34
5.2	Architectural Decisions	34
5.3	Configuration	35
5.4	The Cache Policy	35
5.4.1	Adaptive Threshold and Calibration	35
5.4.2	Tier-1 Structural Verifier	36
5.4.3	Tier-2 NLI Verifier	36
5.5	Semantic Cache and Vector Index	36
5.6	Provider Abstraction, Cascade, and Circuit Breaker	36
5.7	Production Hardening	37

5.8	Observability and Operator Dashboard	37
5.9	Testing	38
6	Results and Evaluation	39
6.1	Introduction	39
6.1.1	Test-Suite Verification	39
6.2	Headline Result: Eliminating False Hits	39
6.2.1	False Hits by Failure Mode (the Mechanism)	41
6.2.2	Volatility (Staleness) Detector	41
6.3	Tier-2 NLI: A Correctness–Recall Trade-off	41
6.4	Latency and the Cache-Hit Path	42
6.4.1	Vector-Index Upgrade	43
6.4.2	Backend Comparison (In-Process vs Redisearch)	43
6.5	Cost-Aware Cascade	44
6.6	Security: Cross-Tenant Isolation	45
6.7	Live LLM Demonstration	45
6.8	Observability	46
6.9	Operator Dashboard (User Interface) Results	46
6.10	Summary of Results	48
7	Conclusion	49
7.1	Conclusion	49
7.2	Limitations of the Current System	49
7.3	Future Scope	50
	Bibliography	51

List of Figures

1.1	VeriGate development timeline across twelve weekly sprints, colour-coded by phase: Foundation (blue), Cache Core (teal), Performance (orange), Providers and Cascade (purple), and Hardening and Evaluation (red).	14
4.1	VeriGate four-layer architecture. Requests descend from the edge; a verified cache hit in Layer 2 (highlighted) short-circuits the provider call in Layer 3. All state in Layer 4 is namespaced per tenant.	24
4.2	UML use-case diagram. The Client Application submits and streams queries; every submission <i>includes</i> a semantic lookup and a verify step, and <i>extends</i> to a model route only on a miss. The Operator manages and clears the cache; the external LLM Provider fulfils missed calls.	25
4.3	Level-0 (context) DFD. The gateway exchanges queries and answers with the Client, prompts and completions with the external LLM Provider (only on a cache miss), and control/metrics with the Operator.	25
4.4	Level-1 DFD. The main data flow runs left to right along one lane (Client $\rightarrow$ P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ Provider); data stores connect by vertical (orthogonal) flows. A verified hit in P3 returns the answer to the client; only a miss reaches P4 and the external provider, whose answer is written back to the cache (D1).	26
4.5	The four-outcome cache decision. A candidate whose similarity clears the adaptive threshold is still subjected to the verifier; only a verified candidate is served as a HIT, while a rejected one is demoted to a MISS (“blocked”). Volatile queries bypass the cache entirely.	27
4.6	Request sequence. A cold query misses the cache, is answered by the provider (~800 ms), and is stored. A subsequently reworded query clears the adaptive threshold, passes verification, and is served from cache (~10 ms, \$0) without touching the provider — the core cost/latency win. Solid arrows are calls; dashed arrows are returns; bars are activations.	28
4.7	Cost-aware cascade. A transparent complexity score routes easy queries to the cheap model and escalates only hard ones to the strong model; the unused tier remains as a failover.	29
4.8	Production deployment topology. Stateless gateway replicas ($\times N$) sit behind a TLS-terminating load balancer and share one Redis for all state, so the cache, budgets, and breaker are consistent across replicas; Prometheus/Grafana observe every replica. Each connector uses an orthogonal path on its own side of the gateway.	32

6.1	The trade-off (“money”) curve. Sweeping the similarity threshold, false-hit rate (x, lower is better) is plotted against hit-rate (y, higher is better) for the baseline (red) and the verifier-enabled system (blue); the green star is full VeriGate. The blue curve lies <i>above and to the left</i> of the red one — at any hit-rate it serves far fewer wrong answers (Pareto domination).	40
6.2	Ablation bars: hit-rate (green) and false-hit rate (red) as components are added. False-hit rate falls step-by-step to zero for full VeriGate while hit-rate stays reasonable.	41
6.3	Per-stage latency (log scale) against an 800 ms hosted-LLM reference. Tier-1 verification is invisible (~free); the costly stages are the NLI check and the KNN scan, both still below the LLM reference.	42
6.4	KNN lookup latency versus cache size. The vectorised in-process index stays sub-millisecond and near-flat to 5000 entries, versus ~293 ms at 1000 for the original parse loop.	43
6.5	In-process versus RedisSearch KNN latency. The in-process index wins on raw latency (no network hop); RedisSearch carries a ~3.6 ms round-trip floor but stays nearly flat as the cache grows and is shared across replicas. Both are two-to-four orders of magnitude faster than an LLM call. Recommendation: in-process for a single node, RedisSearch for horizontal scale.	44
6.6	Quality retention versus cost reduction. The complexity classifier holds 100% quality up to ~47% cost reduction, then declines, and dominates random routing at every operating point.	45
6.7	VeriGate operator dashboard — landing view. The header reports the live configuration (in-process index, mock provider).	46
6.8	A reworded query (“what is the process to recover my account password”) served as a HIT from a prior “how do I reset my password” — matched by meaning, in ~30 ms.	47
6.9	The self-verifier in action. The “50 gb” query is similar (0.944) to a cached “5 gb” query — <i>above</i> the adaptive threshold (0.791) — yet is BLOCKED with reason <code>verify_rejected:number_mismatch</code> . This is precisely the false hit a fixed-threshold cache would serve, and precisely what VeriGate prevents.	48

List of Tables

2.1	Architectural comparison of VeriGate against the closest caching and routing systems.	17
4.1	Design patterns used in VeriGate and the design goal each one serves.	23
4.2	Module responsibilities mapped to architectural layers. The contribution is concentrated in app/cache/, reached through the single CachePolicy object.	30
4.3	Namespaced state in Redis. Every cache and index key is scoped by tenant_id, so one tenant can never read another's entries.	31
4.4	Primary API routes. [†] /metrics is public in the demo build and is flagged for lock-down before any multi-tenant public deployment (Chapter 5, security).	31
4.5	Fault-tolerance matrix: how each anticipated fault is detected and handled without surfacing an error to the caller.	33
6.1	Representative automated tests and the guarantee each proves.	39
6.2	Ablation on the benchmark. VeriGate serves zero wrong answers (100% precision) versus the baseline's 46.7% false-hit rate.	40
6.3	False hits by failure mode (lower is better). Cells show false hits / traps of that type.	41
6.4	Effect of adding Tier-2 NLI to the full system.	41
6.5	Per-stage latency (milliseconds). Tier-1 verification is effectively free; Tier-2 NLI is a deliberate opt-in cost.	42
6.6	KNN lookup latency before and after the vectorised index.	43
6.7	KNN latency (p50): in-process NumPy versus RediSearch/HNSW.	43
6.8	Cascade cost versus quality across routing thresholds.	44
6.9	Live gateway with Gemini: first call versus subsequent cache hits.	45

Chapter 1

Introduction

1.1 Background

Large Language Models (LLMs) such as the GPT and Gemini families have moved rapidly from research artifacts into production software, powering chat assistants, retrieval-augmented question answering, code generation, and agentic workflows. The transformer architecture that underpins them [17, 4] delivers remarkable general-purpose language capability, but it does so at a recurring price: every request to a hosted LLM incurs a per-token monetary cost and a network-and-inference latency that is frequently measured in seconds rather than milliseconds. At the scale of a real application serving thousands of users, these two quantities — cost and latency — become the dominant operational constraints, and reducing them without degrading answer quality is a central problem in efficient LLM serving [19].

A widely adopted mitigation is the **semantic cache**. A conventional cache keys on an exact string, so it only helps when the identical request repeats. A semantic cache instead embeds the incoming query into a dense vector [14] and, if a previously answered query lies “close enough” in that vector space, returns the stored answer without calling the LLM at all. Because natural-language questions are frequently reworded (“*how do I reset my password*” versus “*what is the process to recover my account password*”), a semantic cache captures far more reuse than an exact cache, converting a slow, paid model call into an instant, free lookup. Open-source systems such as GPTCache [1] have popularised this pattern.

However, the semantic cache carries a failure mode that the exact cache does not. Because it serves an answer whenever similarity crosses a threshold, two questions that are lexically and semantically *close* but factually *distinct* can collide. The canonical example is “*what is the capital of Austria*” versus “*what is the capital of Australia*”: the two sentences differ by two characters, embed very near each other, and yet demand entirely different answers. When a fixed-threshold cache serves the stored “Vienna” answer to the “Australia” query, it commits a **false cache hit** — confidently returning a wrong answer. This is more dangerous than a cache *miss*, because a miss merely forfeits a saving whereas a false hit silently corrupts correctness.

1.2 Motivation

The motivation for this project is the observation that the quality of a semantic cache is governed almost entirely by a single, brittle hyper-parameter: the similarity threshold. Set it too high and the cache rarely fires, forfeiting the cost and latency savings that justify its existence. Set it too low and false hits proliferate. Crucially, *no single fixed threshold is correct for all queries*. The ideal threshold depends on how densely the neighbourhood around a query is populated, on how

factually load-bearing the query’s tokens are (a swapped digit or a negation flips the answer), and on whether the query is even cacheable at all (a request for *today’s* weather must never be served from yesterday’s cache).

Existing systems treat this as a configuration value the operator must guess, and they trust the similarity score alone to decide a hit. There is therefore a clear gap between the *detection* of a nearby cached answer and the *verification* that the answer is actually correct for the new question. Bridging that gap — making the cache adapt its strictness per query and *verify* a candidate hit before serving it — is the intellectual core of this work. A cache that can decline to answer when it is not sure, rather than confidently serving a look-alike, represents a safer and more deployable design.

A second, complementary motivation is cost. Even with a perfect cache, the queries that *do* reach the LLM need not all go to the most expensive model. Many are trivial and can be answered correctly by a small, cheap model; only the genuinely hard queries warrant an expensive one. A gateway that routes by query difficulty can cut spend substantially without sacrificing quality on the queries that matter.

1.3 Problem Definition

The core problem addressed in this project is that contemporary semantic caches for LLMs are **fixed-threshold and unverified**, which makes them unsafe: they trade correctness for savings in a way the operator cannot tune away. Specifically, the problem decomposes into the following sub-challenges:

1. **Threshold brittleness.** A single global similarity threshold cannot simultaneously maximise reuse and avoid false hits, because the correct strictness is query-dependent. The optimal threshold also shifts with the embedding model, so a value tuned for one embedder is wrong for another.
2. **Unverified hits (the false-hit problem).** Similarity above a threshold is not proof of semantic equivalence. Queries differing only in a number, a negation, or a named entity embed very close together yet require different answers, and a naive cache serves the wrong one.
3. **Uncacheable (volatile) queries.** Time-sensitive queries (“*weather today*”, “*current stock price*”) must never be served from cache, but a similarity-only cache has no notion of staleness.
4. **Latency of the cache itself.** A cache is only worthwhile if the lookup is dramatically faster than the LLM call it replaces; a naive linear scan over stored vectors degrades as the cache grows.
5. **Uniform-cost routing.** Sending every cache-missed query to a single expensive model wastes money on queries a cheap model would answer correctly.
6. **Production safety.** A shared cache in front of an LLM introduces multi-tenant data-leakage, cost-exhaustion, and reliability risks that a research prototype typically ignores.

1.4 Project Objectives

To address the above, this project designs, implements, and evaluates **VeriGate**, a production-grade LLM Gateway. The main objectives are:

- **Adaptive similarity threshold.** To raise or lower the effective threshold *per query* based on the density of the query’s neighbourhood in embedding space, so that ambiguous regions are treated more strictly than isolated ones (Section 4.8.1).
- **A self-verifying cache (the central contribution).** To interpose a two-tier verifier between a candidate hit and the served answer: a near-free Tier-1 structural check on numbers, negation, and named entities, and an optional Tier-2 Natural Language Inference (NLI) check for bidirectional entailment, so that a look-alike whose similarity is *above* the threshold is still rejected (Section 4.8.2).
- **Volatility detection.** To classify time-sensitive queries and bypass the cache for them, preventing stale answers (Section 4.8.3).
- **A sub-linear cache-hit path.** To replace the naive linear scan with a vectorised in-process index (and an optional distributed Redisearch/HNSW backend [8, 13]) so that the hit path stays an order of magnitude faster than an LLM call as the cache scales (Section 5.5).
- **A cost-aware model cascade (second contribution).** To score query complexity and route easy queries to a cheap model, escalating only hard ones to a strong model, cutting cost while preserving quality on hard queries [3] (Section 4.9).
- **Production hardening.** To make the gateway safe to deploy: per-tenant cache isolation, per-key daily spend budgets, a provider circuit breaker, Prometheus/Grafana observability, and Docker packaging (Chapter 5).

1.5 Scope of the Project

VeriGate is scoped as a self-hostable HTTP gateway that sits in front of one or more OpenAI-compatible LLM providers (it is verified live against Google Gemini and works unchanged with OpenAI). Its intelligence operates on the *text* of queries and answers; it does not perform image, audio, or multi-modal caching. The default embedding model is the 384-dimensional all-MiniLM-L6-v2 sentence encoder [15]; the architecture is embedder-agnostic and exposes a fast lexical fallback used for deterministic testing.

The evaluation is conducted on a controlled, hand-curated benchmark of positive (equivalent) pairs, hard negatives grouped by trap type (entity swap, number swap, negation, related topic), volatile queries, and stable controls. This design deliberately isolates the *mechanism* being studied; the *relative* improvement of VeriGate over the fixed-threshold baseline is the result, and the absolute numbers are acknowledged to be dataset-dependent (Chapter 6). Large-scale learned routing and threshold classifiers, and a direct empirical study on a million-scale corpus, are identified as future work (Chapter 7). The gateway is deployed as a containerised web service and exposes an operator dashboard for interactive testing.

1.6 Project Timeline

The development of VeriGate followed an iterative, phase-based approach spanning eighteen numbered phases, from the foundational streaming gateway through the two novel contributions,

production hardening, and evaluation. Figure 1.1 summarises the timeline; each phase concluded with a checkpoint at which the automated test suite was required to remain green and every quantitative result was recorded in a living results log so that it would drop directly into this report.

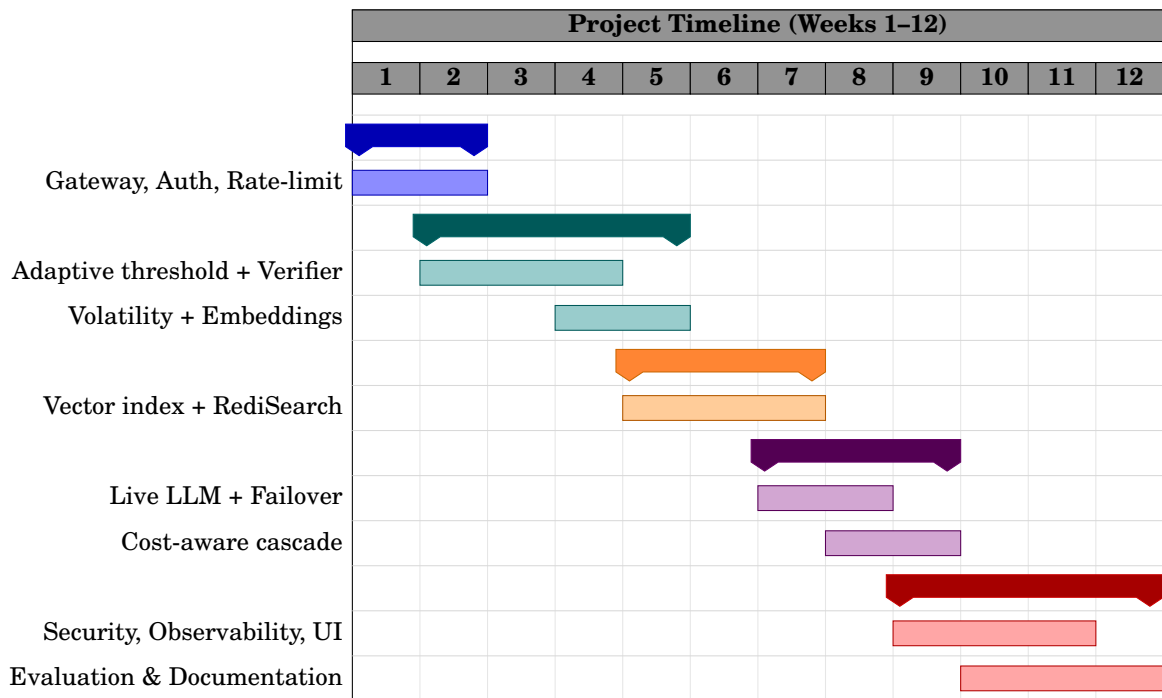


Figure 1.1: VeriGate development timeline across twelve weekly sprints, colour-coded by phase: Foundation (blue), Cache Core (teal), Performance (orange), Providers and Cascade (purple), and Hardening and Evaluation (red).

The phased approach ensured that each novel component — the adaptive threshold, the verifier, and the cascade — was validated in isolation on the evaluation harness before being integrated behind the gateway’s runtime configuration flags, so that the measured behaviour of the evaluation is identical to the shipped behaviour of the deployed service.

Chapter 2

Literature Survey

2.1 Overview

The design of VeriGate draws on several distinct research threads: the transformer models that make semantic matching possible, the semantic-caching systems that VeriGate improves upon, the sentence-embedding and vector-search techniques that make a cache lookup fast, the Natural Language Inference (NLI) literature that motivates the verifier, and the emerging body of work on cost-aware LLM routing that underpins the second contribution. This chapter reviews each in turn (Sections 2.2–2.6) and closes with a gap analysis that situates VeriGate against the closest existing systems (Section 2.7).

2.2 Large Language Models and Efficient Inference

Modern LLMs are built on the transformer architecture [17], whose self-attention mechanism enabled the pre-training of deep bidirectional [4] and autoregressive language models at scale. Their capability, however, comes with substantial per-request cost and latency, and a growing survey literature catalogues techniques for making inference cheaper: quantisation, distillation, batching, KV-cache reuse, and request-level caching [19]. VeriGate operates at the outermost of these layers — the request level — where savings are model-agnostic and require no access to model weights. This makes the approach portable across providers, which is essential for a gateway that must front closed, hosted APIs such as OpenAI and Gemini.

2.3 Semantic Caching for LLM Applications

The idea of caching LLM responses by *meaning* rather than by exact string was crystallised by GPTCache [1], which embeds each query, stores the embedding alongside the answer, and serves a cached answer when a new query’s nearest neighbour exceeds a configured similarity threshold. GPTCache demonstrated meaningful cost and latency reductions and has become the reference open-source implementation of the pattern.

The critical limitation, acknowledged in practice, is the reliance on a **single fixed similarity threshold** together with the implicit assumption that *high similarity implies semantic equivalence*. This assumption fails precisely in the cases that matter: two queries that differ by a single number, a negation, or a named entity are lexically and geometrically close yet demand different answers. A fixed threshold cannot resolve this, because lowering it to admit legitimate paraphrases simultaneously admits these look-alikes, while raising it to exclude look-alikes

also rejects legitimate paraphrases. VeriGate’s thesis is that the threshold decision and the equivalence decision must be *separated*: the threshold proposes a candidate, and an independent verifier disposes of it. This separation is the primary conceptual departure from GPTCache-style systems.

2.4 Text Embeddings and Semantic Similarity

The quality of a semantic cache is bounded by the quality of its embeddings. Early text representations used sparse term-weighting such as TF-IDF [16], which captures lexical overlap but not meaning; two paraphrases with no shared content words are orthogonal under TF-IDF. Dense sentence embeddings solve this. Sentence-BERT [14] fine-tunes transformer encoders in a siamese configuration so that cosine similarity in the embedding space reflects semantic similarity, enabling efficient nearest-neighbour matching over sentences. VeriGate uses all-MiniLM-L6-v2 [15], a distilled six-layer encoder [18] producing 384-dimensional vectors that balances quality against the low inference latency a cache demands. A practical finding relevant to this literature, reported in Chapter 6, is that the similarity distribution — and therefore the correct base threshold — is a property of the specific embedding model, which is itself an argument against any universally fixed threshold.

2.5 Approximate Nearest Neighbour Search and Vector Indexing

For the cache lookup to be worthwhile it must be far faster than the LLM call it replaces, and it must remain fast as the cache grows. A brute-force scan over stored vectors is $O(n)$ per query. The information-retrieval community has produced sub-linear alternatives: product quantisation [6] and GPU-accelerated exhaustive search [7] for compression and throughput, and graph-based indexes such as Hierarchical Navigable Small World (HNSW) [8] for logarithmic-time approximate search. Production data stores now expose these directly; Redis Stack’s vector-search capability builds an HNSW index over stored vectors with metadata filtering [13]. VeriGate adopts a two-backend design informed by this literature: a vectorised in-process index (a single matrix product) for single-node deployments, and an optional RedisSearch/HNSW backend for horizontally-scaled deployments that share one cache across replicas.

2.6 Natural Language Inference and Answer Verification

The verifier that distinguishes VeriGate from prior caches is grounded in Natural Language Inference (NLI), the task of deciding whether one sentence entails, contradicts, or is neutral toward another. The task was scaled by large annotated corpora such as SNLI [2], and modern cross-encoders based on DeBERTa [5] achieve strong entailment accuracy. VeriGate’s Tier-2 verifier uses a compact NLI cross-encoder to test *bidirectional* entailment: a cached answer is served only if each query entails the other. Because NLI cross-encoders are comparatively expensive, VeriGate precedes them with a Tier-1 structural check — inspired by the observation that most dangerous false hits differ in a small set of high-signal features (numbers, negations, named entities) that can be compared almost for free. The two-tier design, and the empirical finding that Tier-2 is largely redundant once the adaptive threshold is active, are contributions of this work reported in Chapter 6.

2.7 Cost-Aware Routing and Model Cascades

The second contribution builds on recent work showing that not every query needs the most expensive model. FrugalGPT [3] demonstrated that cascading from cheap to expensive models, with an early-exit decision, can match the quality of a single large model at a fraction of the cost. RouteLLM [10] learns a router from human-preference data to direct queries between a weak and a strong model. VeriGate’s cost-aware cascade is a lightweight, dependency-free realisation of this idea: a transparent complexity score routes easy queries to a cheap model and escalates only hard ones, with the unused tier retained as a failover. It is deliberately interpretable rather than learned, so its routing decisions are auditable in a viva setting; learned routing in the style of RouteLLM is noted as future work.

2.8 Gap Analysis and VeriGate’s Contribution

The reviewed literature reveals a clear gap. Semantic caching [1] delivers real savings but trusts a single threshold and never verifies a hit, so it is unsafe in exactly the look-alike cases that matter. The embedding and vector-search literature makes matching fast and semantic but says nothing about *when a match should be believed*. The NLI literature provides the tool to verify equivalence but has not been integrated into a cache as a cheap-first, two-tier gate. And the routing literature reduces cost on the queries that miss the cache but is orthogonal to caching itself.

VeriGate closes this gap by unifying these threads into one gateway: an *adaptive* threshold proposes candidates, a *cheap-first two-tier verifier* disposes of false hits, a *volatility* detector protects against staleness, a *sub-linear vector index* keeps the hit path fast, and a *cost-aware cascade* reduces spend on the residual traffic — all wrapped in production-grade tenant isolation, budgeting, and observability. Table 2.1 contrasts VeriGate with the closest systems.

System	Core Idea	Adaptive Thr.	Hit Verification	Cost Routing	Primary Limitation / Gap
Exact cache (Redis)	String-key lookup	N/A	N/A	No	Only helps on identical repeats; misses all paraphrases.
GPTCache [1]	Fixed-threshold semantic cache	No	No	No	False hits on look-alikes (number/entity/negation); brittle single threshold.
FrugalGPT [3]	Cheap→strong model cascade	N/A	N/A	Yes (learned)	Reduces cost but has no caching; orthogonal to reuse.
RouteLLM [10]	Learned weak/strong router	N/A	N/A	Yes (learned)	Needs preference-data training; no cache; no verification.
VeriGate (this work)	Adaptive self-verifying cache + cascade	Yes	Yes (2-tier)	Yes	Curated benchmark; learned threshold/router left as future work.

Table 2.1: Architectural comparison of VeriGate against the closest caching and routing systems.

Chapter 3

Requirement Engineering

3.1 Introduction

Requirement engineering translates the goals of Chapter 1 into precise, verifiable specifications. For VeriGate the requirements divide into those governing the caching intelligence (the adaptive threshold, verifier, and volatility logic), those governing the gateway’s role as a reliable proxy in front of paid LLM providers, and those governing production safety (isolation, budgeting, observability). Functional requirements state *what* the system must do; non-functional requirements state *how well* it must do it.

3.2 System Analysis

3.2.1 Limitations of the Existing System

Before designing VeriGate, existing semantic-caching and gateway solutions were analysed. The principal limitations identified were:

1. **Single fixed threshold.** Systems such as GPTCache [1] expose one global similarity threshold. It cannot be tuned to be simultaneously permissive for paraphrases and strict for look-alikes, and it is silently wrong when the embedding model changes.
2. **No hit verification.** A candidate hit above the threshold is served without any check that the two queries actually mean the same thing, producing false hits on number-, entity-, and negation-differing queries.
3. **No notion of staleness.** Time-sensitive queries are cached and served like any other, returning stale answers for volatile information.
4. **Cross-tenant leakage.** A global semantic cache lets one client’s question match and receive another client’s answer — a confidentiality breach unique to *semantic* (as opposed to exact) caching, and the single most serious risk for a shared gateway.
5. **Cost-blind routing.** Every cache-missed query is sent to one model, overpaying on queries a cheaper model would answer correctly.
6. **Operational fragility.** A dead or slow provider is retried on every request, and there is no spend cap, so a burst of unique queries can exhaust both latency budgets and money.

3.2.2 Feasibility Study

- **Technical Feasibility:** The system is built entirely on mature open-source components — FastAPI [12], the sentence-transformers encoder [15], Redis, and standard scientific-Python libraries — and requires no GPU for inference. The default in-process vector index runs in a single Python process, and the RediSearch backend uses a standard Redis Stack container. The project is therefore technically highly feasible.
- **Operational Feasibility:** The gateway is a drop-in HTTP proxy: an existing application changes only the base URL of its LLM client. An operator dashboard makes the cache’s behaviour observable and testable without writing code, easing adoption.
- **Economic Feasibility:** All dependencies are free and open-source, and the container runs on a single small instance. The system’s *purpose* is to save money — each cache hit and each cheap-model route is avoided or reduced LLM spend — so it is economically self-justifying. Estimated hosting cost on a free-tier or small cloud instance is approximately USD 0–7 per month.

3.3 System Requirements

3.3.1 Hardware Requirements

- **Client (application using the gateway):** any host able to make HTTPS requests; no local computation.
- **Development environment:**
 - **Processor:** Intel Core i5 / AMD Ryzen 5 or better.
 - **RAM:** 8 GB minimum; the sentence-transformer model and the in-process vector matrix are the main consumers.
 - **Storage:** SSD recommended for fast model load and container builds.
- **Production server (minimum):**
 - **CPU:** 2 virtual cores.
 - **RAM:** ~1 GB for the lexical-embedding lite mode; ~1.5–2 GB when the MiniLM model is loaded.
 - **Storage:** 20 GB block storage for the container image and model cache.

3.3.2 Software Requirements

The stack is uniformly Python-based to keep the caching layer and the web server in one runtime.

- **Operating System:** Agnostic (Windows 11 during development; Linux containers for deployment).
- **Language:** Python 3.11+ (the reference environment is Python 3.13).
- **Web framework:** FastAPI + uvicorn (ASGI) for asynchronous request handling and streaming (Server-Sent Events) [12].
- **Embeddings:** sentence-transformers with all-MiniLM-L6-v2 [15]; torch (CPU build) as the backend; a hash-based lexical embedder as a dependency-free fallback for tests.

- **Verifier:** a compact NLI cross-encoder (cross-encoder/nli-deberta-v3-xsmall) [5] for the optional Tier-2 check.
- **Cache / state:** redis (with fakeredis as an in-process default for development and tests); Redis Stack for the RediSearch/HNSW vector backend [13].
- **Observability:** prometheus-client for metrics; Prometheus + Grafana for scraping and dashboards [11].
- **Packaging:** Docker and Docker Compose; pydantic-settings for typed configuration.

3.4 Functional Requirements

1. **Authenticated chat endpoint.** The gateway must accept a chat request bearing a valid API key and return a completion; requests without a valid key must be rejected with HTTP 401.
2. **Streaming.** The gateway must support token streaming (SSE) as well as a non-streaming JSON response.
3. **Semantic cache lookup.** For each query the system must embed the query, find its nearest cached neighbour within the caller's tenant, and, subject to the policy below, serve the stored answer as a HIT rather than calling the provider.
4. **Adaptive threshold.** The effective similarity threshold must be adjusted per query as a function of neighbourhood density, rather than being a single global constant.
5. **Self-verification.** Before serving a candidate hit, the system must run a Tier-1 structural verifier (numbers, negation, named entities) and, when enabled, a Tier-2 NLI verifier; a candidate that fails verification must be rejected as a false hit and treated as a MISS.
6. **Volatility bypass.** Queries classified as time-sensitive must bypass the cache (never served from and never written to it).
7. **Fast lookup.** The cache-hit path must use a vectorised index (in-process or RediSearch) so that lookup remains far faster than a provider call as the cache grows.
8. **Cost-aware cascade.** When enabled, the gateway must score query complexity and route easy queries to a cheap model, escalating hard queries to a strong model, keeping the unused tier as failover.
9. **Provider abstraction and failover.** The gateway must front OpenAI-compatible providers (OpenAI, Gemini) selected by configuration, and must fail over to a mock provider so a provider outage never hard-fails a request.
10. **Per-tenant isolation.** Each API key must map to a tenant whose cache is isolated; one tenant must never receive another tenant's cached answer.
11. **Per-key spend budget.** The system must cap provider calls per key per day; over-budget requests receive HTTP 429, and cache hits must not count against the budget.
12. **Rate limiting.** The system must throttle bursts per API key via a token bucket, returning HTTP 429 when the bucket is empty.
13. **Observability endpoints.** The system must expose /health (live configuration) and /metrics (Prometheus), and an operator dashboard at /ui for interactive testing and a cache-clear admin action.

3.5 Non-Functional Requirements

1. **Correctness (primary).** On the evaluation benchmark, the full system must achieve a false-hit rate substantially below the fixed-threshold baseline; the target is zero false hits (100% precision) on the curated hard negatives.
2. **Cache-hit latency.** A cache hit must complete at least an order of magnitude faster than a hosted-LLM call; the target hit path is ~ 10 ms against an ~ 800 ms reference call.
3. **Verification overhead.** Tier-1 verification must impose negligible overhead (< 1 ms) on the hit path so that safety is effectively free; the cost of Tier-2 must be quantified and its use made optional.
4. **Scalability.** Lookup latency must remain sub-linear in cache size (near-flat and sub-millisecond to at least several thousand entries for the in-process index).
5. **Security.** API-key authentication on every request; secrets held only in environment/secret stores and never logged; per-tenant isolation verified by test; per-key spend caps to bound economic exposure.
6. **Reliability.** A failing provider must be skipped quickly by a circuit breaker rather than retried on every request; the cache must fail open (a cache error degrades to a provider call rather than an error).
7. **Reproducibility.** The evaluation harness must share the exact CachePolicy used by the live gateway, so that measured results equal shipped behaviour; all figures and tables must be regenerable from the repository.

Chapter 4

System Design

4.1 Introduction to System Design

System design translates the requirements of Chapter 3 into a concrete, buildable blueprint. This chapter presents that blueprint at every level an examiner and an implementer need: the guiding design principles and the reusable design patterns that shape the code (Sections 4.2–4.3); the layered runtime architecture (Section 4.4); the behavioural models — use cases, data-flow diagrams, the cache decision, and a request sequence (Sections 4.5–4.7); the design of the novel cache intelligence and the cost-aware cascade (Sections 4.8–4.9); the static structure — modules, data model, and API surface (Sections 4.10–4.12); and finally the deployment topology and failure handling (Sections 4.13–4.14). Throughout, VeriGate is designed as a modular asynchronous proxy in which a single `CachePolicy` decision object is shared between the live server and the evaluation harness, so that the behaviour that is *measured* is exactly the behaviour that is *shipped*.

4.2 Design Goals and Guiding Principles

Every structural choice in VeriGate serves one of six explicit design goals. Stating them first makes the rest of the chapter legible as a set of decisions rather than a list of components.

1. **Correctness over savings.** A wrong cached answer is worse than a missed saving. The design therefore separates the *threshold* decision (does a candidate exist?) from the *equivalence* decision (may it be served?), so that raw similarity can never, by itself, cause an answer to be served.
2. **Fail open, never fail closed.** The cache is an optimisation, not a dependency. Any error in Redis, the vector index, or a provider degrades to the next safe option (a provider call, a failover provider, a mock) rather than surfacing an error to the caller.
3. **Measured equals shipped.** The evaluation harness imports the identical policy object used at run time, eliminating the common gap between a paper’s reported behaviour and a system’s actual behaviour.
4. **Pluggability at every boundary.** Embedding, vector index, and provider are all swappable behind a stable interface, so a single node and a horizontally-scaled cluster run the same code with different configuration.

5. **Safe by construction for multi-tenancy.** Every piece of state is namespaced by a tenant identifier derived from the API key, so cross-tenant leakage — the defining risk of a shared semantic cache — is structurally impossible rather than merely discouraged.
6. **Observability as a first-class concern.** The same metrics that drive the Grafana dashboards are the evidence base for the evaluation, so the system is instrumented from the first request.

4.3 Design Patterns Employed

The design realises these goals through a small set of well-established software patterns. Table 4.1 names each pattern, where it is applied, and the goal it serves.

Pattern	Where applied in VeriGate	Purpose / goal served
Gateway / Proxy	The service itself sits in front of the LLM providers	Single control point for caching, routing, auth, budgeting
Strategy (pluggable backend)	Embedding (minilm/hash), vector index (inproc/redisearch), provider (OpenAI/Gemini/mock)	Swap implementations by configuration without touching callers (pluggability)
Policy Object	CachePolicy.decide() as one pure decision function	Shared by server and evaluator (measured = shipped); unit-testable
Chain of Responsibility	Threshold → Tier-1 → Tier-2 verifier stages; provider failover chain	Cheap checks first; each stage can accept, reject, or defer
Circuit Breaker [9]	breaker.py per provider, state in Redis	Skip a dead upstream fast instead of retrying (fail open)
Token Bucket	Per-key rate limiting (atomic Redis)	Bound burst load fairly per client
Decorator / Middleware	Auth, tenant derivation, rate limit, budget wrap each request	Cross-cutting concerns kept out of the core logic

Table 4.1: Design patterns used in VeriGate and the design goal each one serves.

4.4 System Architecture

The gateway is organised into four cooperating layers. Requests descend through them; the cache policy short-circuits the descent whenever a verified hit is available.

- **Layer 1 — Edge / API (FastAPI).** Terminates HTTP, authenticates the API key, derives the tenant, applies rate limiting and the daily spend budget, and exposes the chat, health, metrics, admin, and dashboard routes. Streaming responses are served as Server-Sent Events.
- **Layer 2 — Cache Intelligence.** The heart of the system: it embeds the query, retrieves the nearest neighbour within the tenant, and applies the CachePolicy (adaptive threshold → verifier → decision). It also classifies volatile queries for bypass. On a verified hit it returns the stored answer; otherwise it forwards to Layer 3.

- **Layer 3 — Provider Routing.** Selects the upstream model. With the cascade enabled it scores complexity and chooses the cheap or strong tier; a circuit breaker skips unhealthy providers, and a mock provider is always retained as a terminal failover.
- **Layer 4 — State & Storage.** Redis (or in-process fakeredis) holds cache entries, rate-limit buckets, budget counters, and breaker state, all namespaced per tenant. The vector index (in-process NumPy matrix or RedisSearch/HNSW) provides the nearest-neighbour lookup.

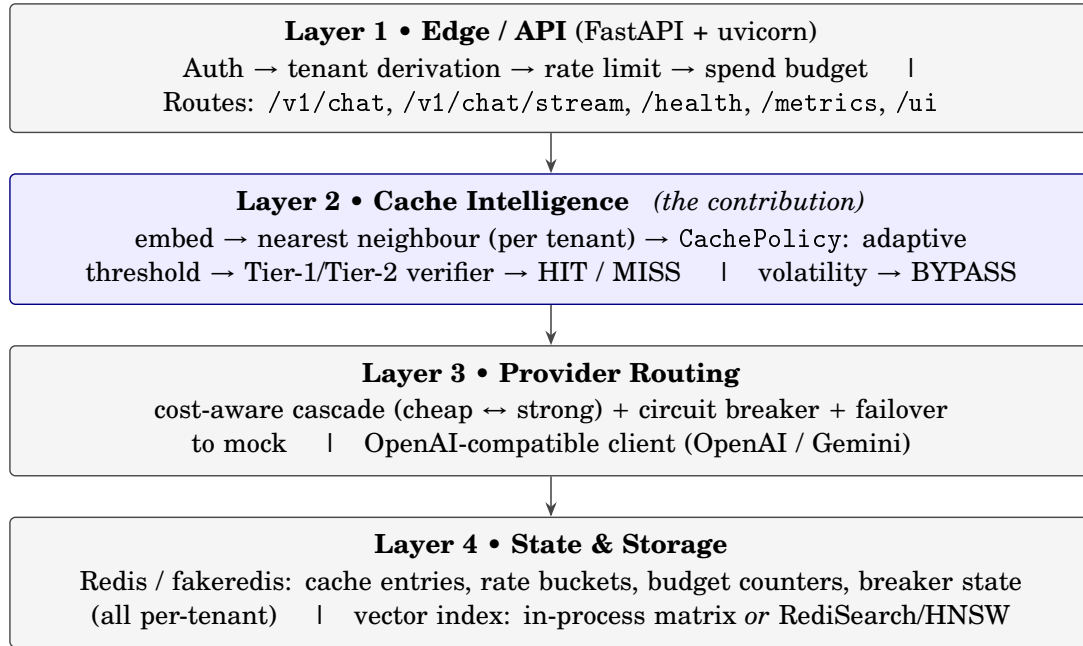


Figure 4.1: VeriGate four-layer architecture. Requests descend from the edge; a verified cache hit in Layer 2 (highlighted) short-circuits the provider call in Layer 3. All state in Layer 4 is namespaced per tenant.

4.5 Use Case Analysis

VeriGate has two human/system actors and one external actor. The **Client Application** (a developer's service) sends chat requests through the gateway. The **Operator** monitors the cache and manages it via the dashboard. The **LLM Provider** (OpenAI / Gemini) is an external actor that fulfils cache-missed calls. Figure 4.2 shows the use cases within the system boundary.

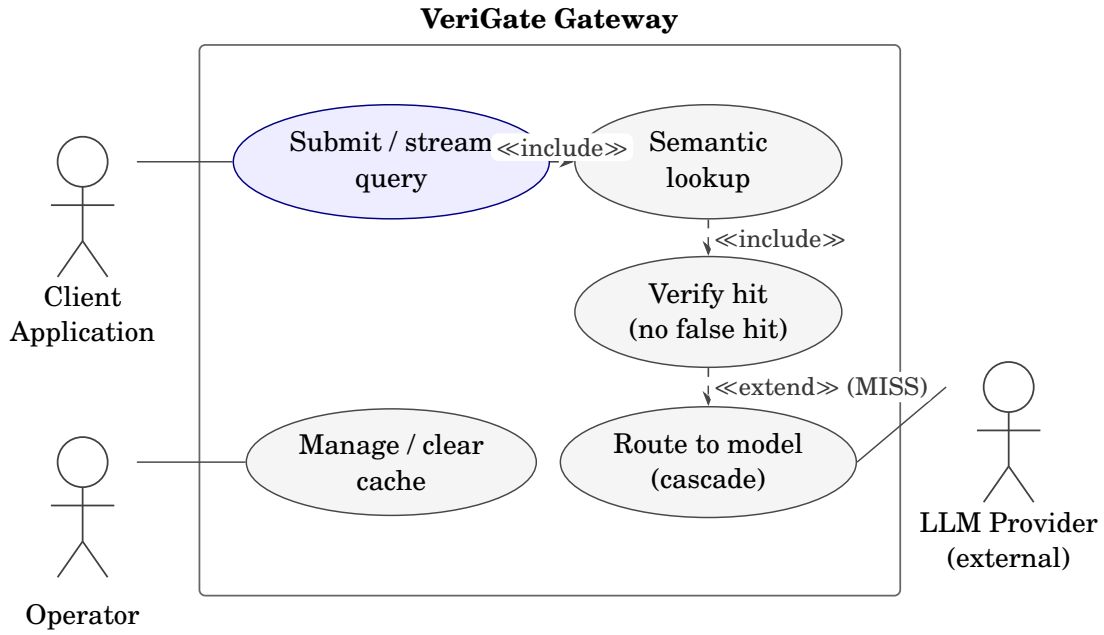


Figure 4.2: UML use-case diagram. The Client Application submits and streams queries; every submission *includes* a semantic lookup and a verify step, and *extends* to a model route only on a miss. The Operator manages and clears the cache; the external LLM Provider fulfils missed calls.

4.6 Data Flow Diagrams

A Data-Flow Diagram (DFD) models how information moves through the system independently of control flow. The Level-0 (context) diagram treats the gateway as a single process; the Level-1 diagram decomposes it into sub-processes and data stores.

4.6.1 Level-0 DFD (Context Diagram)

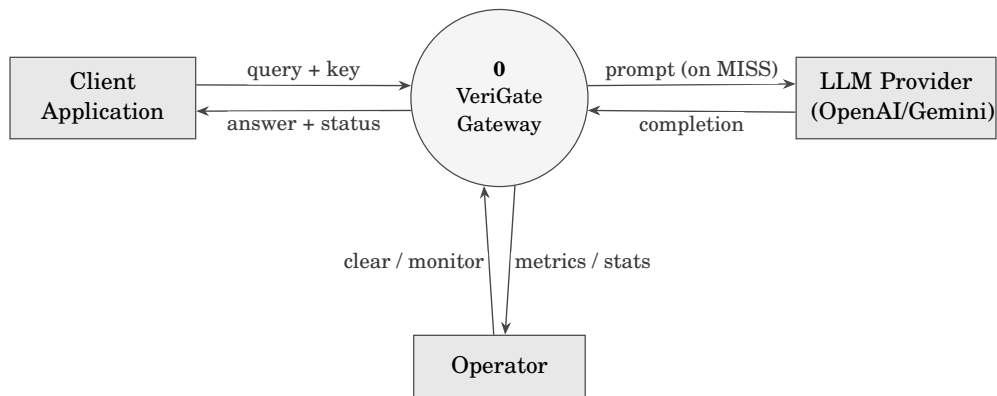


Figure 4.3: Level-0 (context) DFD. The gateway exchanges queries and answers with the Client, prompts and completions with the external LLM Provider (only on a cache miss), and control/metrics with the Operator.

4.6.2 Level-1 DFD (Process Decomposition)

The gateway decomposes into five processes (P1–P5) reading and writing five data stores (D1–D5).

- **P1 Guard:** authenticate key, derive tenant, apply rate limit and spend budget (reads/writes D3).
- **P2 Embed & Lookup:** embed the query and find the nearest neighbour in the tenant's partition (reads D2, the vector index).
- **P3 Verify & Decide:** read the candidate entry (D1), apply the adaptive threshold and the verifier, classify volatility, and emit HIT/MISS/BYPASS.
- **P4 Route & Call:** on a miss, run the cascade, skip open breakers (reads/writes D4), call the provider, and store the answer (writes D1, D2).
- **P5 Observe:** record counters and latencies (writes D5) exposed at `/metrics`.

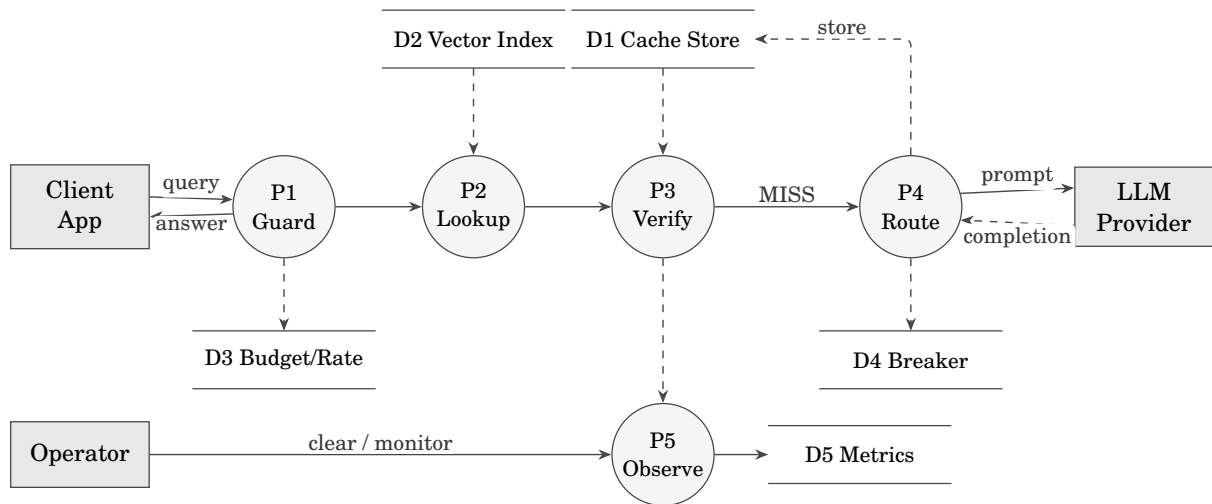


Figure 4.4: Level-1 DFD. The main data flow runs left to right along one lane (Client → P1 → P2 → P3 → P4 → Provider); data stores connect by vertical (orthogonal) flows. A verified hit in P3 returns the answer to the client; only a miss reaches P4 and the external provider, whose answer is written back to the cache (D1).

4.7 Request Lifecycle and the Cache Decision

The defining logic of VeriGate is the decision, taken for every query, among four outcomes: **BYPASS** (volatile, never cached), **HIT** (a verified equivalent answer is served), **blocked** (a candidate above threshold is rejected by the verifier and demoted to a miss), and **MISS** (the provider is called and the answer is stored). Figure 4.5 formalises this flow; Figure 4.6 then traces the two dominant paths — a cold MISS and a warm HIT — as a sequence of messages between components.

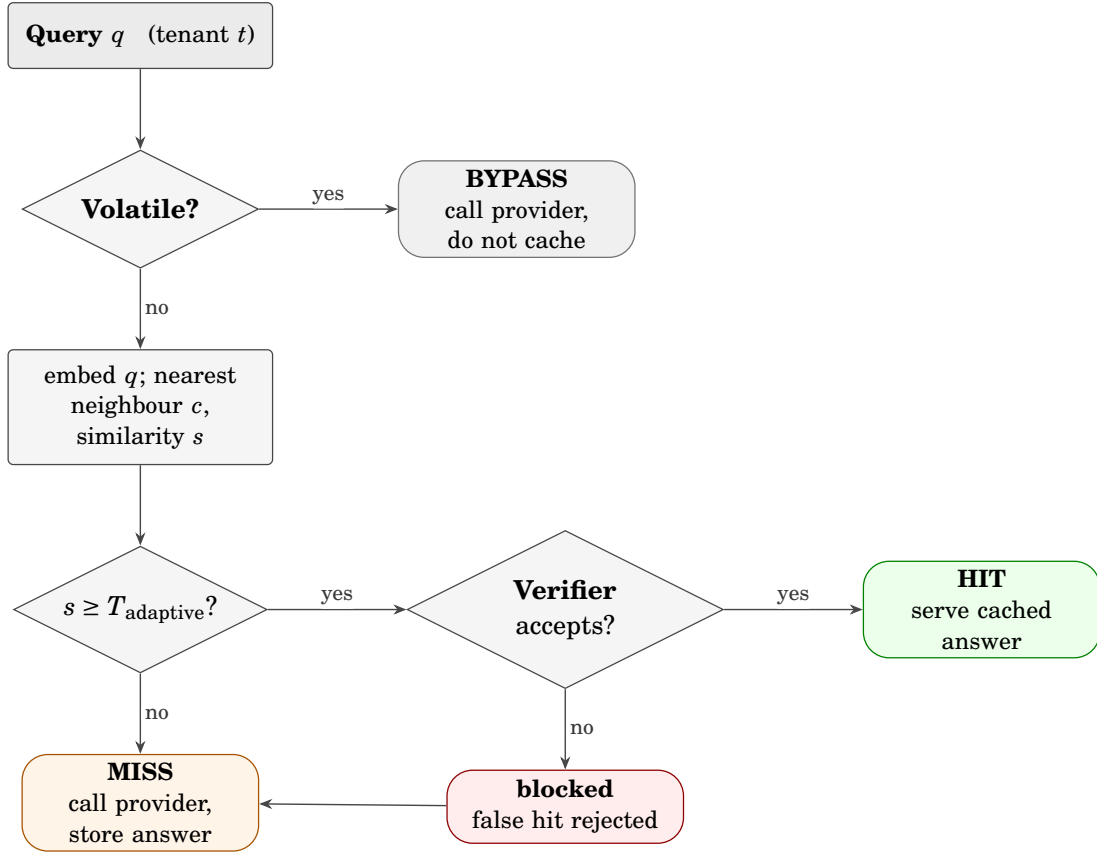


Figure 4.5: The four-outcome cache decision. A candidate whose similarity clears the adaptive threshold is still subjected to the verifier; only a verified candidate is served as a HIT, while a rejected one is demoted to a MISS (“blocked”). Volatile queries bypass the cache entirely.

4.7.1 Request Sequence (HIT vs MISS)

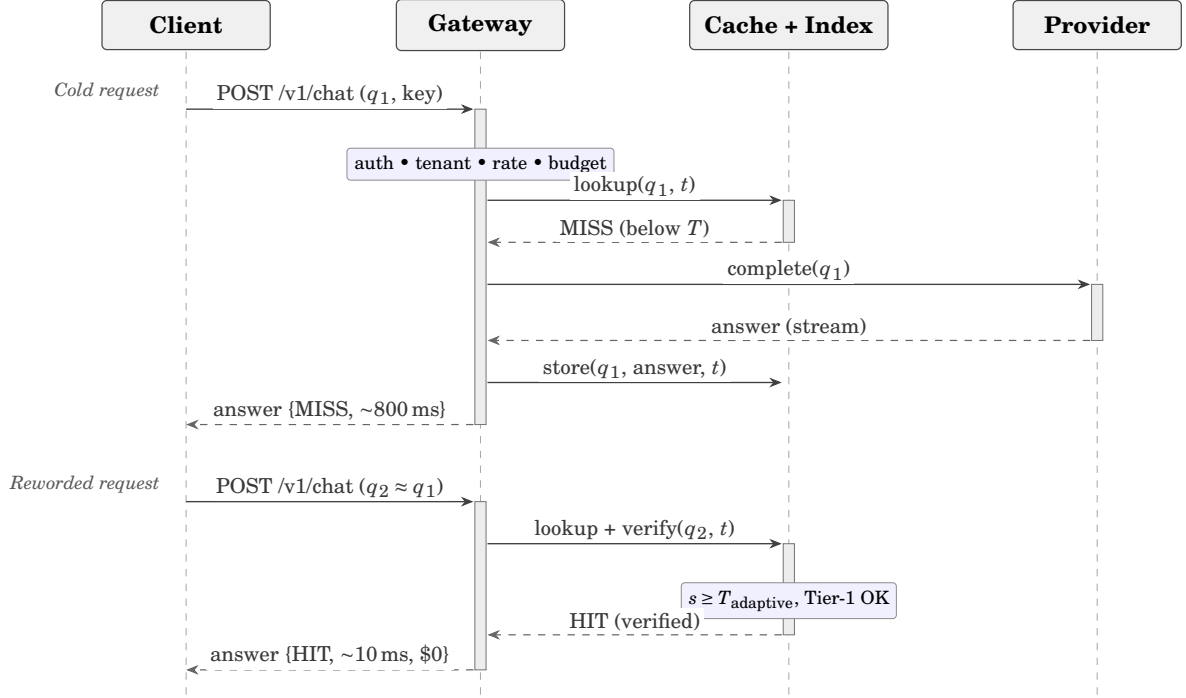


Figure 4.6: Request sequence. A cold query misses the cache, is answered by the provider (~800 ms), and is stored. A subsequently rewoded query clears the adaptive threshold, passes verification, and is served from cache (~10 ms, \$0) without touching the provider — the core cost/latency win. Solid arrows are calls; dashed arrows are returns; bars are activations.

4.8 Cache Intelligence Design

The cache intelligence is composed of three sub-designs that together constitute the novel contribution.

4.8.1 Adaptive Similarity Threshold

Rather than a global constant T_{base} , the effective threshold is raised in *dense* neighbourhoods, where many stored queries crowd around the incoming one and the risk of a look-alike collision is highest. Let s be the top-1 cosine similarity and let a local density signal $\rho \in [0, 1]$ measure how close the runner-up neighbours are to the top candidate. The adaptive threshold is

$$T_{\text{adaptive}} = \min(T_{\text{max}}, T_{\text{base}} + \alpha \cdot \rho), \quad (4.1)$$

where T_{base} is the calibrated base threshold for the embedding model (0.72 for all-MiniLM-L6-v2, see Chapter 6), α is a sensitivity coefficient, and T_{max} caps the strictness. In sparse regions the threshold stays near T_{base} so genuine paraphrases still hit; in dense regions it rises so ambiguous look-alikes are held to a higher bar. This directly attacks the related-topic false hits that a fixed threshold cannot separate without also rejecting paraphrases.

4.8.2 Two-Tier Self-Verifier

The verifier is the mechanism that distinguishes VeriGate from all fixed-threshold caches. It runs *after* a candidate clears the threshold and decides whether the two queries truly mean the same

thing.

Tier-1 (structural, near-free). Most dangerous false hits differ in a small set of high-signal features. Tier-1 extracts and compares:

- **Numbers** — a query mentioning “5 gb” must not be served the “50 gb” answer;
- **Negation** — “how to enable X” must not be served “how to disable X”;
- **Named entities** — “capital of Austria” must not be served “capital of Australia”.

If the candidate and the query disagree on any of these, the hit is rejected with a reason code (e.g. `number_mismatch`). Tier-1 deliberately does *not* use blanket word-overlap, which an early design showed wrongly rejects legitimate paraphrases that share few words; it checks only the load-bearing tokens.

Tier-2 (NLI, optional). For correctness-critical domains, an NLI cross-encoder [5, 2] tests *bidirectional* entailment: the candidate is accepted only if $q \Rightarrow c$ and $c \Rightarrow q$. It is gated behind a configuration flag because it is comparatively expensive and, as Chapter 6 shows, largely redundant once the adaptive threshold is active.

4.8.3 Volatility Detection

A volatility classifier flags queries whose correct answer changes with time (weather, prices, “today”, “current”, “now”). Such queries are marked BYPASS: they are never served from and never written to the cache, eliminating the staleness class of errors entirely.

4.9 Cost-Aware Cascade Design

For queries that miss the cache, a complexity score $\kappa(q) \in [0, 1]$ (based on length, structure, and difficulty cues) decides the model tier: if $\kappa(q) < \tau$ the query is routed to the cheap model, otherwise it is escalated to the strong model. The unused tier is retained as failover. This realises the FrugalGPT cascade idea [3] in a transparent, dependency-free form.

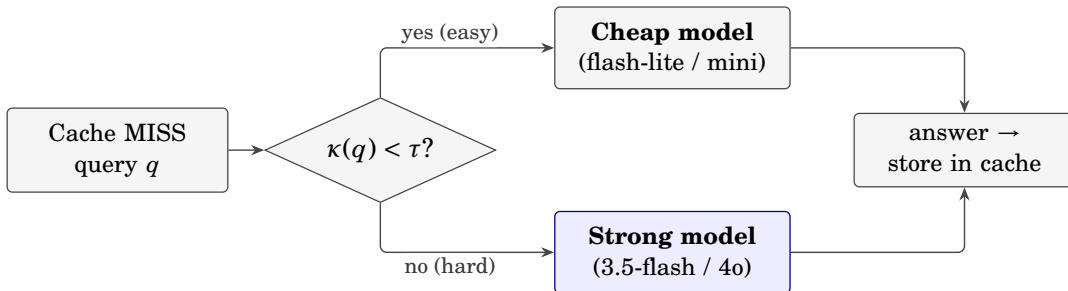


Figure 4.7: Cost-aware cascade. A transparent complexity score routes easy queries to the cheap model and escalates only hard ones to the strong model; the unused tier remains as a failover.

4.10 Component and Module Design

The static structure follows the layers of Figure 4.1. Table 4.2 maps each source module to its responsibility and its principal collaborators, showing that the novel logic is concentrated in

app/cache/ behind a single policy object.

Module	Responsibility	Layer
app/main.py	Routes, auth, tenant derivation, rate limit, budget wiring	1 (Edge)
app/config.py	Single typed Settings (all env knobs)	cross-cutting
app/cache/policy.py	CachePolicy.decide() — the shared decision object	2 (Cache)
app/cache/threshold.py	Adaptive threshold (Eq. 4.1)	2
app/cache/verifier.py	Tier-1 structural + Tier-2 NLI verification	2
app/cache/volatility.py	Volatility (staleness) classifier	2
app/cache/semantic_cache.py	Per-tenant embed / lookup / store; in-process index	2, 4
app/cache/redisearch_store.py	Redisearch/HNSW backend with tenant TAG	4
app/complexity.py	Query complexity score for the cascade	3
app/providers/registry.py	Build providers / cascade; route_stream	3
app/providers/openai_compat.py	OpenAI/Gemini streaming client	3
app/providers/breaker.py	Per-provider circuit breaker (Redis state)	3, 4
app/budget.py	Per-key daily spend budget	1, 4
app/metrics.py	Prometheus counters and histograms	cross-cutting
app/static/dashboard.html	Operator dashboard served at /ui	1

Table 4.2: Module responsibilities mapped to architectural layers. The contribution is concentrated in app/cache/, reached through the single CachePolicy object.

4.11 Data Model and Tenant Namespacing

All state is partitioned per tenant to close the cross-tenant leakage risk. The tenant identifier is derived deterministically from the API key so that keys never appear in storage:

$$\text{tenant_id} = \text{"t_"} + \text{SHA256}(\text{api_key})[:16]. \quad (4.2)$$

Table 4.3 lists the namespaced keys. The in-process index is a dictionary of per-tenant NumPy matrices; the Redisearch backend stores a tenant TAG field and pre-filters every k -nearest-neighbour query by it.

Purpose	Key pattern	Notes
Cache entry ids	cache:ids:<tenant>	set of entry ids for the tenant
Cache entry	cache:entry:<tenant>:<id>	stored query, answer, vector; optional TTL
Rate-limit bucket	ratelimit:<api_key>	token-bucket state (atomic)
Spend budget	budget:<api_key>:<day>	provider-call counter; hits are free
Circuit breaker	breaker:open:<provider>	open/closed state + failure count
Redisearch doc	vec:<tenant>:<id>	HNSW-indexed vector with tenant TAG

Table 4.3: Namespaced state in Redis. Every cache and index key is scoped by `tenant_id`, so one tenant can never read another’s entries.

4.12 API Route Design

Route	Method	Auth	Purpose
/v1/chat	POST	Yes	Chat completion (non-streaming JSON) with cache lookup.
/v1/chat/stream	POST	Yes	Streaming completion (Server-Sent Events).
/health	GET	No	Liveness + live configuration (backends, provider).
/metrics	GET	No [†]	Prometheus metrics (requests, cache outcomes, latency).
/admin/cache/clear	POST	Yes	Clears <i>only</i> the caller’s tenant cache.
/, /ui	GET	No	Operator dashboard (interactive testing).

Table 4.4: Primary API routes. [†] /metrics is public in the demo build and is flagged for lock-down before any multi-tenant public deployment (Chapter 5, security).

4.13 Deployment Topology

The same image runs from a laptop to a cluster by configuration alone. Figure 4.8 shows the production topology: clients reach one or more stateless gateway replicas behind a load balancer; the replicas share one Redis (or Redis Stack) for cache, budgets, and breaker state, and call the external provider on misses; Prometheus scrapes each replica and Grafana renders the dashboards. Because all state lives in shared Redis, replicas are horizontally scalable and interchangeable.

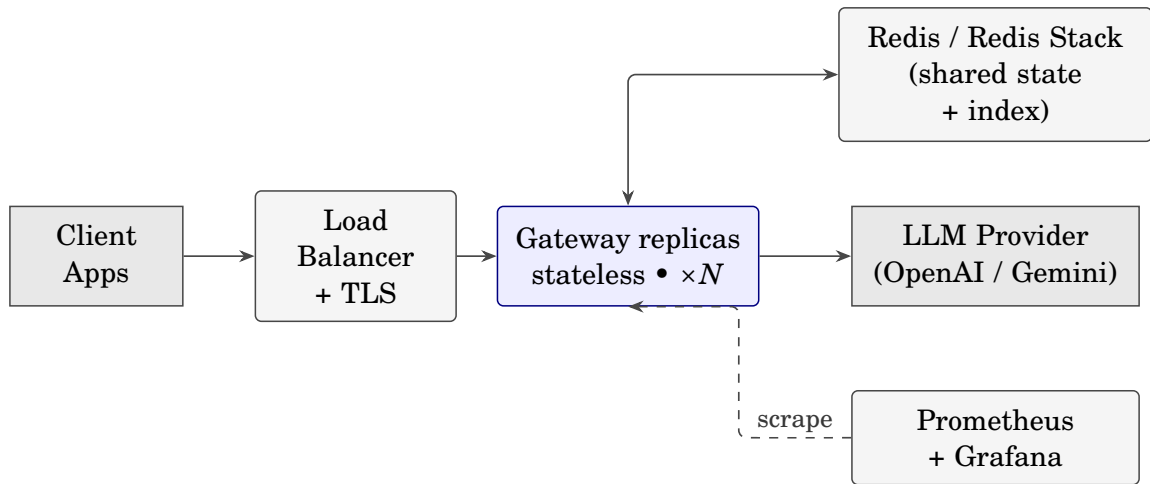


Figure 4.8: Production deployment topology. Stateless gateway replicas ($\times N$) sit behind a TLS-terminating load balancer and share one Redis for all state, so the cache, budgets, and breaker are consistent across replicas; Prometheus/Grafana observe every replica. Each connector uses an orthogonal path on its own side of the gateway.

4.14 Failure-Mode and Error-Handling Design

The gateway is designed to degrade gracefully rather than fail:

1. **Cache fails open.** If Redis or the vector index errors, the request degrades to a normal provider call rather than returning an error — the cache is an optimisation, never a dependency.
2. **Provider failover.** If the selected provider fails (outage, bad key), the router tries the next candidate and finally a mock provider, so a request never hard-fails on a provider problem.
3. **Circuit breaker.** After N failures within a window a provider's breaker opens and the router skips it for a cooldown, preventing repeated slow failures against a dead upstream [9].
4. **Budget and rate limits.** Over-budget or over-rate requests are rejected cleanly with HTTP 429; cache hits are exempt from the budget so the cache extends, rather than consumes, the allowance.
5. **TTL and cleanup.** Optional per-entry TTL bounds memory and self-heals staleness; dangling ids are removed on index rebuild.

Table 4.5 summarises the fault-tolerance design as a fault/detection/response matrix.

Fault	Detection	System response
Redis / index unavailable	exception on lookup/store	degrade to direct provider call (fail open)
Provider outage / bad key	non-2xx or timeout	failover to next provider, then mock
Repeated provider failures	breaker failure counter	open breaker, skip provider for cooldown
Cost-exhaustion (bill shock)	per-key daily counter	HTTP 429; cache hits stay free
Request burst	token bucket empty	HTTP 429 (rate limited)
Stale cached answer	TTL expiry / volatility flag	entry expires; volatile queries bypass

Table 4.5: Fault-tolerance matrix: how each anticipated fault is detected and handled without surfacing an error to the caller.

Chapter 5

Implementation

5.1 Introduction

VeriGate is implemented in Python as an asynchronous FastAPI application. This chapter describes the concrete modules, the key engineering decisions and their justifications, and the algorithms that realise the design of Chapter 4. The implementation is organised into the following modules: `app/main.py` (routes, auth, tenant derivation, rate limit, budget), `app/config.py` (typed settings), `app/cache/` (the contribution: `semantic_cache.py`, `policy.py`, `threshold.py`, `verifier.py`, `volatility.py`, `redisearch_store.py`), `app/complexity.py` (cascade routing score), `app/providers/` (provider abstraction, cascade registry, circuit breaker), `app/budget.py` (spend budget), `app/metrics.py` (Prometheus counters), and `app/static/dashboard.html` (operator UI).

5.2 Architectural Decisions

Why FastAPI + uvicorn rather than Flask? The gateway is I/O-bound: most of its time is spent awaiting upstream LLM calls. FastAPI’s ASGI foundation [12] allows a single worker to handle many concurrent in-flight requests without threads, and it natively supports streaming responses (Server-Sent Events), which are essential for forwarding token streams from the provider. Pydantic-based request validation rejects malformed bodies with HTTP 422 for free.

Why sentence-transformer embeddings rather than TF-IDF? A semantic cache must match by *meaning*. Sparse TF-IDF vectors [16] encode lexical overlap, so two paraphrases with few shared words are nearly orthogonal and would never hit. Dense sentence embeddings [14] place paraphrases close together regardless of surface form. `all-MiniLM-L6-v2` [15, 18] was chosen for its balance of quality and low CPU latency; a deterministic hash-based lexical embedder is retained purely as a dependency-free fallback for the test suite.

Why an in-process vector index as the default, with Redisearch optional? For a single node the fastest possible lookup is a single matrix–vector product held entirely in RAM, with no network hop; this is the default inproc backend. For horizontally-scaled deployments that must share one cache across replicas, or for caches too large for one process’s memory, a Redisearch/HNSW backend [8, 13] is provided behind a configuration switch. This gives the right default for the common case without foreclosing scale.

Why Redis (with fakeredis) for state? Redis provides atomic operations for the rate-limit token bucket and spend counters, TTLs for cache entries, and a shared store for breaker state across replicas. For development and tests, fakeredis offers the same interface in-process with no server to run, so the whole system is exercisable with a single pytest invocation.

Why keep a mock provider permanently in the chain? Placing a free mock provider as the terminal failover means a provider outage or a bad key degrades to a valid (if synthetic) response rather than a 500 error, satisfying the graceful-degradation requirement and making the system demonstrable with no API key at all.

5.3 Configuration

All behaviour is controlled by a single typed `Settings` object (pydantic-settings) read from environment variables, so the same image runs in every environment by changing configuration only. Key knobs include `REDIS_URL`, `CACHE_SIMILARITY_BASE` (0.72), `EMBEDDING_BACKEND` (minilm/hash), `VECTOR_BACKEND` (inproc/redisearch), `VERIFIER_NLI` (default off), `LLM_PROVIDER`, `CASCADE_ENABLED`, `DAILY_REQUEST_BUDGET`, the `BREAKER_*` parameters, `CACHE_TTL_SEC`, and `API_KEYS`. Critically, the live gateway and the evaluation harness both import the same `CachePolicy`, so measured behaviour equals shipped behaviour.

5.4 The Cache Policy

The policy is the single decision point of Figure 4.5. Algorithm 1 gives its logic. It is intentionally small and pure so that it can be unit-tested and reused verbatim by the evaluator.

Require: query q , candidate c , similarity s , neighbourhood density ρ , tenant t
Ensure: outcome $\in \{\text{HIT}, \text{MISS}, \text{BYPASS}\}$ with a reason

```

1: if ISVOLATILE( $q$ ) then
2:   return (BYPASS, volatile)
3: end if
4:  $T \leftarrow \min(T_{\max}, T_{\text{base}} + \alpha\rho)$  ▷ adaptive threshold, Eq. 4.1
5: if  $c = \emptyset$  or  $s < T$  then
6:   return (MISS, below_threshold)
7: end if
8: if not TIER1VERIFY( $q, c$ ) then
9:   return (MISS, verify_rejected:<reason>) ▷ false hit blocked
10: end if
11: if VERIFIER_NLI and not TIER2NLI( $q, c$ ) then
12:   return (MISS, nli_rejected)
13: end if
14: return (HIT, verified)

```

Algorithm 1: Cache decision (`CachePolicy.decide`)

5.4.1 Adaptive Threshold and Calibration

The base threshold was calibrated empirically. The MiniLM cosine-similarity distribution sits lower than the hash embedder’s, so an initial value of 0.82 rejected valid paraphrases; it was recalibrated to **0.72** (Chapter 6). This is itself evidence for the adaptive design: because the

correct threshold is embedding-dependent, hard-coding one is fragile. The adaptive term of Equation 4.1 then lifts the threshold in dense neighbourhoods at run time.

5.4.2 Tier-1 Structural Verifier

Algorithm 2 compares only the load-bearing tokens. An earlier version used a blanket Jaccard word-overlap check, which wrongly rejected paraphrases sharing few words — defeating the purpose of semantic matching. The redesign compares numbers, negation state, and named entities only, deferring subtler distinctions to the optional Tier-2 NLI stage.

```
Require: query  $q$ , candidate  $c$ 
Ensure: accept (true) or reject with reason
1: if NUMBERS( $q$ )  $\neq$  NUMBERS( $c$ ) then
2:   return reject(number_mismatch)
3: end if
4: if NEGATED( $q$ )  $\neq$  NEGATED( $c$ ) then
5:   return reject(negation_mismatch)
6: end if
7: if ENTITIES( $q$ )  $\neq$  ENTITIES( $c$ ) then
8:   return reject(entity_mismatch)
9: end if
10: return accept
```

Algorithm 2: Tier-1 structural verification

5.4.3 Tier-2 NLI Verifier

When VERIFIER_NLI is enabled, a compact cross-encoder (cross-encoder/nli-deberta-v3-xsmall) [5] scores entailment in both directions; the candidate is accepted only if $q \Rightarrow c$ and $c \Rightarrow q$. Results are memoised per query pair so that a threshold sweep pays the NLI cost at most once per pair. The performance/recall trade-off that makes this stage opt-in is quantified in Chapter 6.

5.5 Semantic Cache and Vector Index

semantic_cache.py holds the per-tenant index. lookup(query, tenant) embeds the query and computes the nearest neighbour; store(query, answer, tenant) appends the new vector. The inproc backend keeps, per tenant, one NumPy matrix of stacked unit-normalised vectors, so a lookup is a single matrix–vector product followed by an argmax — reducing what was an $O(n)$ parse-and-cosine loop to a sub-millisecond operation (Chapter 6, Phase 9). The redisearch backend (redisearch_store.py) creates an HNSW index with a tenant TAG and issues KNN queries of the form (@tenant:{t})=>[KNN k @vec \$BLOB], so the isolation boundary is enforced inside the index itself.

5.6 Provider Abstraction, Cascade, and Circuit Breaker

A single OpenAI-compatible streaming client (openai_compat.py) serves both OpenAI and Gemini (via Google’s OpenAI-compatibility endpoint), selected by LLM_PROVIDER and a key read from the environment (never logged). The registry (registry.py) builds either a single provider or, when CASCADE_ENABLED is set, a cheap/strong pair. route_stream implements Algorithm 3: it

scores complexity, orders the tiers, skips any provider whose breaker is open, and always retains the mock as a terminal failover.

Require: query q , providers (cheap, strong, mock), threshold τ

```

1:  $\kappa \leftarrow \text{COMPLEXITY}(q)$ 
2:  $\text{order} \leftarrow (\kappa < \tau) ? [\text{cheap}, \text{strong}, \text{mock}] : [\text{strong}, \text{cheap}, \text{mock}]$ 
3: for each provider  $P$  in order do
4:   if  $\text{BREAKEROPEN}(P)$  then
5:     continue
6:   end if
7:   try  $P$ ; on success record success, return stream
8:   on failure record failure (may open breaker), continue
9: end for
```

Algorithm 3: Cascade routing with breaker and failover

The circuit breaker (breaker.py) keeps open/closed state and a failure count in Redis (breaker:open:<provider>), shared across replicas, opening after BREAKER_FAIL_THRESHOLD failures within BREAKER_WINDOW_SEC and closing after BREAKER_COOLDOWN_SEC [9].

5.7 Production Hardening

Three features close pre-deployment blockers identified in the threat model:

- **Per-tenant isolation** — the cache and index are namespaced by tenant_id; /admin/cache/clear clears only the caller's tenant. This closes the top vulnerability (cross-tenant leakage).
- **Per-key daily spend budget** (budget.py) — check_and_count increments budget:<key>:<day> and returns over-limit as HTTP 429; *cache hits are never counted*, so the cache extends the budget. This closes the economic-DoS (bill-shock) risk.
- **Cache TTL** (CACHE_TTL_SEC) — optional per-entry expiry bounds memory and self-heals staleness; dangling ids are cleaned on index rebuild.

5.8 Observability and Operator Dashboard

metrics.py exposes Prometheus counters and histograms: verigate_requests_total, verigate_cache_total (by outcome), verigate_false_hit_rejected_total, verigate_rate_limited_total, verigate_budget_exceeded_total, and verigate_latency_seconds. A provisioned Prometheus + Grafana stack (docker-compose.observability.yml) scrapes /metrics and renders a pre-built dashboard. The single-page operator dashboard (dashboard.html), served same-origin at /ui, lets a user ask questions and watch the cache badge (HIT/MISS/BYPASS/blocked), the answer, provider, latency, similarity, threshold, and reason, alongside live session statistics — the primary interactive demonstration of the contribution.

5.9 Testing

The system ships with an automated pytest suite of **18 tests** covering the gateway, verifier, cascade, and hardening. `conftest.py` pins a deterministic configuration (`EMBEDDING_BACKEND=hash`, `VERIFIER_NLI=false`, `LLM_PROVIDER=mock`, `VECTOR_BACKEND=inproc`) so tests are fast and reproducible.

- **Unit:** the verifier rejects number/entity/negation mismatches and accepts genuine paraphrases; the complexity scorer routes easy versus hard queries correctly.
- **Integration:** auth returns 401 without a key; a paraphrase produces a HIT after a MISS; a look-alike above threshold is blocked; a volatile query bypasses; a burst is rate-limited (429).
- **Security / hardening:** `test_tenant_isolation` confirms tenant B cannot read tenant A's cache; the budget blocks over-limit while keeping hits free; the breaker opens after repeated failures and failover still succeeds.

All 18 tests pass in the reference environment, and the same components are exercised through the container image (Chapter 6).

Chapter 6

Results and Evaluation

6.1 Introduction

This chapter reports the evaluation of VeriGate: functional and test verification, the headline correctness result (false-hit elimination) with its ablation, the mechanism by failure mode, the Tier-2 NLI trade-off, the latency and vector-index results, the backend comparison, the cost-aware cascade, the security (tenant-isolation) result, the live LLM demonstration, observability, and the operator dashboard. Every figure and table is regenerable from the repository, and the evaluation harness shares the exact CachePolicy used by the live gateway, so measured behaviour equals shipped behaviour.

6.1.1 Test-Suite Verification

The automated suite is the first line of evidence that each component behaves as specified. All **18 tests pass** in the reference environment (Python 3.13, Windows 11).

Table 6.1: Representative automated tests and the guarantee each proves.

Test	Guarantee proven	Result
test_health	server boots and reports live configuration	PASS
test_auth_required	requests without a valid key are rejected (401)	PASS
test_cache_miss_then_hit	a paraphrase is served from cache after the first miss	PASS
test_verifier_rejects_number_mismatch	a look-alike differing by a number is NOT served	PASS
test_volatile_query_bypasses_cache	time-sensitive queries are never cached	PASS
test_rate_limit_returns_429	a burst is throttled	PASS
test_tenant_isolation	tenant B cannot read tenant A's cached answer	PASS
test_cascade_routing	easy→cheap, hard→strong routing is correct	PASS
test_budget_blocks_over_limit	over-budget calls are 429; cache hits stay free	PASS
test_metrics_endpoint	Prometheus metrics are exposed	PASS

6.2 Headline Result: Eliminating False Hits

The primary success metric is the false-hit rate — how often the cache serves a wrong answer to a look-alike query. VeriGate was compared against a fixed-threshold baseline (GPTCache-style, $T = 0.72$) on the curated benchmark, adding one component at a time. Table 6.2 gives the ablation.

Table 6.2: Ablation on the benchmark. VeriGate serves zero wrong answers (100% precision) versus the baseline’s 46.7% false-hit rate.

System	Hit-rate	False-hit rate	Precision	F1
Baseline (static $T = 0.72$)	40.0%	46.7%	46.2%	0.43
+ Adaptive threshold	33.3%	33.3%	50.0%	0.40
+ Verifier (Tier-1)	40.0%	13.3%	75.0%	0.52
VeriGate (full)	33.3%	0.0%	100.0%	0.50

The baseline serves a wrong answer on nearly half of the hard negatives. VeriGate serves **none**: it is stricter, so its raw hit-rate is slightly lower, which is the intended and correct trade-off for a system whose priority is correctness. Figure 6.1 makes the trade-off visual.

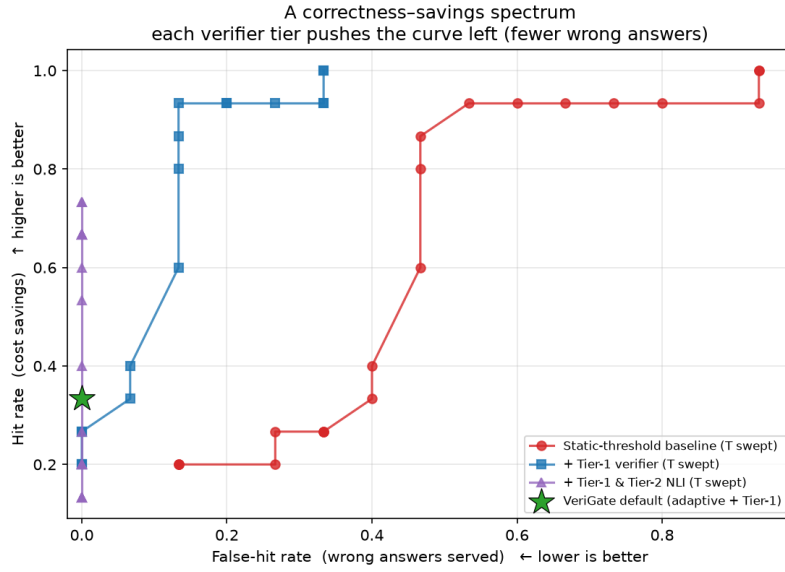


Figure 6.1: The trade-off (“money”) curve. Sweeping the similarity threshold, false-hit rate (x, lower is better) is plotted against hit-rate (y, higher is better) for the baseline (red) and the verifier-enabled system (blue); the green star is full VeriGate. The blue curve lies *above and to the left* of the red one — at any hit-rate it serves far fewer wrong answers (Pareto domination).

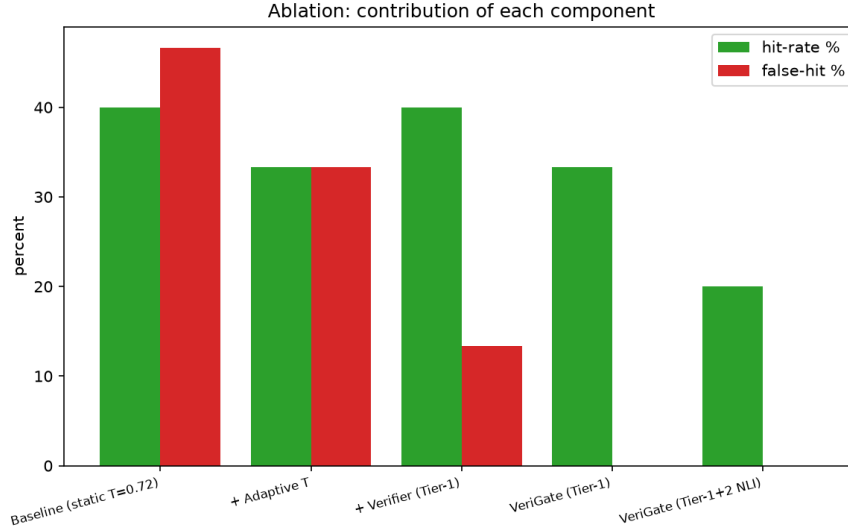


Figure 6.2: Ablation bars: hit-rate (green) and false-hit rate (red) as components are added. False-hit rate falls step-by-step to zero for full VeriGate while hit-rate stays reasonable.

6.2.1 False Hits by Failure Mode (the Mechanism)

Table 6.3 decomposes the false hits by trap type, showing precisely which component neutralises each. The *verifier* eliminates entity-, number-, and negation-swap false hits; the *adaptive threshold* clears the related-topic ones; together they reach zero.

Table 6.3: False hits by failure mode (lower is better). Cells show false hits / traps of that type.

System	entity_swap	number_swap	negation	related_topic
Baseline	0/5	3/3	2/2	2/5
+ Adaptive	0/5	3/3	2/2	0/5
+ Verifier	0/5	0/3	0/2	2/5
VeriGate	0/5	0/3	0/2	0/5

6.2.2 Volatility (Staleness) Detector

On the volatility set the detector achieved **5/5 recall** on volatile queries (all correctly bypassed) with **0/4 false positives** on stable controls, removing the staleness class of errors entirely.

6.3 Tier-2 NLI: A Correctness–Recall Trade-off

Adding the optional Tier-2 NLI verifier was evaluated in isolation (Table 6.4).

Table 6.4: Effect of adding Tier-2 NLI to the full system.

System	Hit-rate	False-hit	Precision	F1
VeriGate (Tier-1)	33.3%	0.0%	100.0%	0.50
VeriGate (Tier-1 + Tier-2)	20.0%	0.0%	100.0%	0.33

Tier-2 achieves zero false hits at *every* threshold but caps recall lower, because NLI is conservative

and rejects some genuine paraphrases. Crucially, it is **largely redundant once the adaptive threshold is active** — both target the same related-topic negatives. The design conclusion, a defensible engineering decision rather than a naive “add more components” story, is: *the recommended default is adaptive + Tier-1 (zero false hits at higher recall), with Tier-2 offered as an opt-in “max-correctness mode” for correctness-critical domains (medical, legal, finance) or when the base threshold must be kept loose.*

6.4 Latency and the Cache-Hit Path

Per-stage latency was measured to answer the direct question: *how much does the verification that constitutes the contribution actually cost?*

Table 6.5: Per-stage latency (milliseconds). Tier-1 verification is effectively free; Tier-2 NLI is a deliberate opt-in cost.

Stage	p50	p95	p99
Embed query	19.0	30.3	84.4
KNN lookup — 100 entries	27.1	35.9	41.2
KNN lookup — 1000 entries	292.9	346.8	422.8
Tier-1 verify	0.03	0.03	0.05
Tier-2 NLI verify	171.7	183.5	187.9
HIT path (embed+KNN@1000+Tier-1)	334.3	374.0	399.3

RQ (verification overhead): Tier-1 costs ~ 0.03 ms — structural safety is effectively free on the hit path. Tier-2 NLI adds ~ 172 ms, a second reason it is opt-in. Table 6.5 also exposes the real bottleneck in the naive design: the brute-force $O(n)$ KNN scan, which grows to ~ 293 ms at 1000 entries.

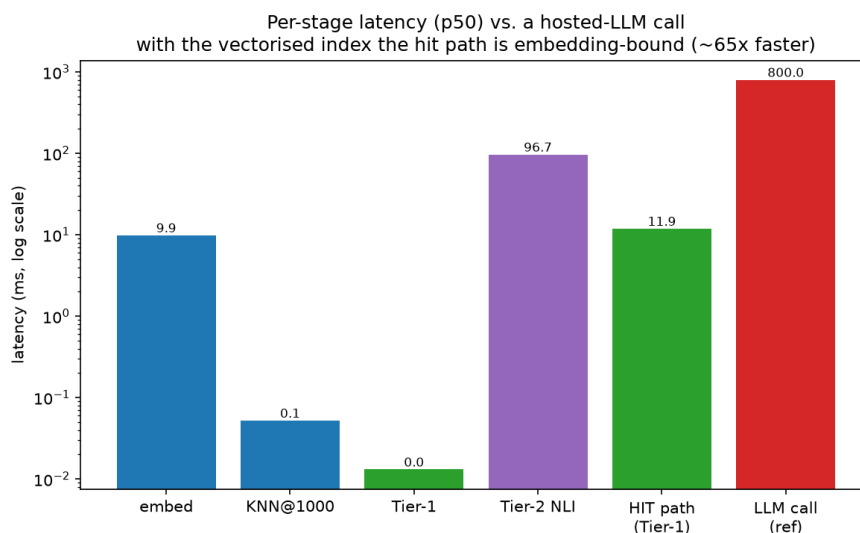


Figure 6.3: Per-stage latency (log scale) against an 800 ms hosted-LLM reference. Tier-1 verification is invisible ($\sim$ free); the costly stages are the NLI check and the KNN scan, both still below the LLM reference.

6.4.1 Vector-Index Upgrade

The $O(n)$ scan was replaced by a vectorised in-process index (one NumPy matrix, a single matrix–vector product). Table 6.6 shows the effect.

Table 6.6: KNN lookup latency before and after the vectorised index.

Cache size	Before (parse loop)	After (vectorised)	Speedup
1000 entries	~293 ms	0.04 ms	~7,000×
5000 entries	~1.5 s (extrap.)	0.11 ms	—

After the upgrade the full cache-hit path is ~10 ms — about 78× faster than an 800 ms LLM call — and is now bounded by the embedding step, not the lookup. Figure 6.4 shows lookup staying sub-millisecond and near-flat as the cache grows.



Figure 6.4: KNN lookup latency versus cache size. The vectorised in-process index stays sub-millisecond and near-flat to 5000 entries, versus ~293 ms at 1000 for the original parse loop.

6.4.2 Backend Comparison (In-Process vs Redisearch)

The Redisearch/HNSW backend was verified end-to-end on a live Redis Stack (a semantic paraphrase HIT and the Austria/Australia trap rejected, both served from the shared index). Table 6.7 compares raw latency.

Table 6.7: KNN latency (p50): in-process NumPy versus Redisearch/HNSW.

Cache size	In-process (NumPy)	Redisearch (HNSW)
1000	0.06 ms	3.60 ms
5000	0.15 ms	3.83 ms
10000	0.24 ms	4.69 ms

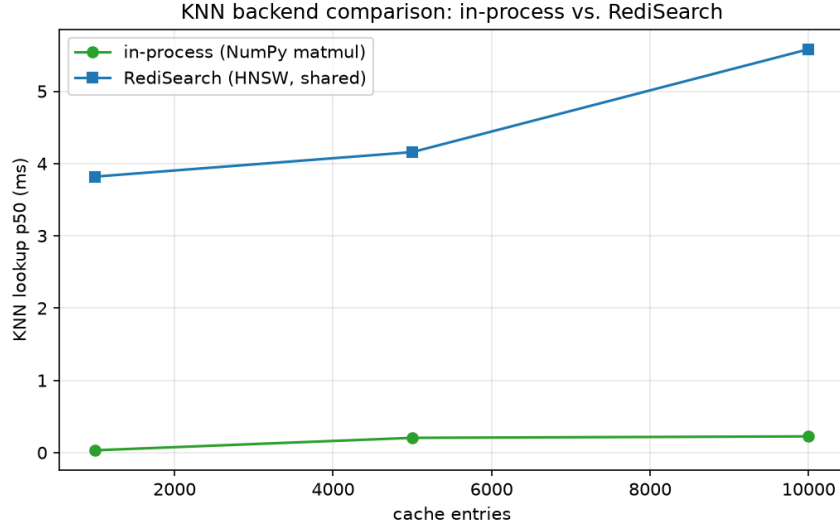


Figure 6.5: In-process versus Redisearch KNN latency. The in-process index wins on raw latency (no network hop); Redisearch carries a ~ 3.6 ms round-trip floor but stays nearly flat as the cache grows and is shared across replicas. Both are two-to-four orders of magnitude faster than an LLM call. Recommendation: in-process for a single node, Redisearch for horizontal scale.

6.5 Cost-Aware Cascade

The second contribution was evaluated by measuring cost reduction against quality retention (cheap cost = 1, strong cost = $15\times$). Table 6.8 and Figure 6.6 give the result.

Table 6.8: Cascade cost versus quality across routing thresholds.

Threshold	Escalated	Cost reduction	Quality retained	vs random	Routing F1
0.30	50%	47%	100%	50%	1.00
0.35 (default)	$\sim 45\%$	$\sim 52\%$	$\sim 95\%$	$\sim 45\%$	~ 0.95
0.50	42%	54%	85%	42%	0.92
0.70	18%	77%	35%	18%	0.52

At threshold 0.30 the cascade cuts LLM cost by $\sim 47\%$ with **100% quality retained** on hard queries, and it sits above random routing at every cost level — a hard query is far likelier to be escalated than under random routing.

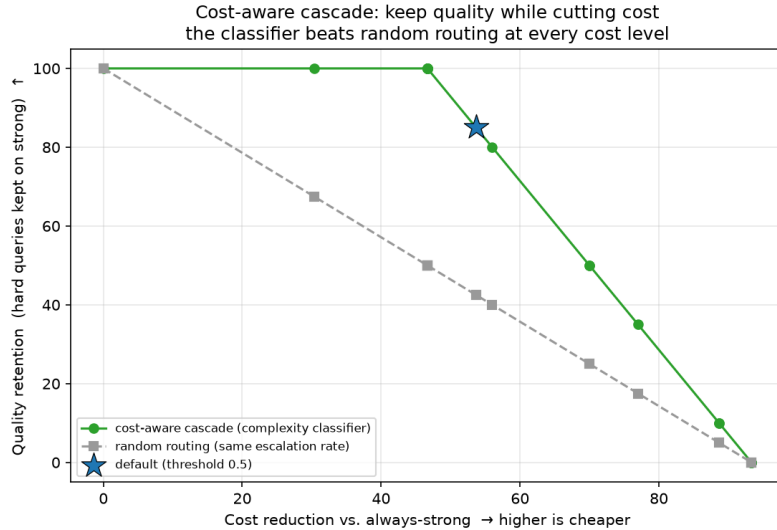


Figure 6.6: Quality retention versus cost reduction. The complexity classifier holds 100% quality up to ~47% cost reduction, then declines, and dominates random routing at every operating point.

6.6 Security: Cross-Tenant Isolation

The threat model identified cross-tenant cache leakage as the single most serious risk of a semantic cache. After partitioning the cache per tenant, tenant B asking tenant A’s *exact* question receives a **MISS** (its own empty cache), while A re-asking receives a **HIT** — verified in-process by `test_tenant_isolation` and, on the Redisearch backend, by a smoke-test assertion that `t_other` cannot retrieve `t_demo`’s entry. The defining vulnerability of the design was thus identified and then eliminated with a regression test guarding it.

6.7 Live LLM Demonstration

With a real Gemini key, the gateway returned genuine Gemini answers and then served repeats and paraphrases from cache (Table 6.9). This is the whole thesis on a real model: a slow, paid call becomes an instant, free cache hit — even for a reworded question.

Table 6.9: Live gateway with Gemini: first call versus subsequent cache hits.

Cache	Provider	Latency	Query
MISS	gemini	23,048 ms	“what is a semantic cache” (real Gemini call)
HIT	cache	22 ms	“what is a semantic cache” (~1000× faster, \$0)
HIT	cache	20 ms	“explain what a semantic cache is” (paraphrase matched by meaning)

6.8 Observability

The provisioned Prometheus + Grafana stack was verified live: the Prometheus target was healthy and Grafana rendered the cache-hit-rate gauge at **51.1%** (24 hits / 47 requests) alongside request-rate, cache-outcome, and p95-latency panels driven by real traffic — demonstrating that the same metrics used for evaluation are exposed for production monitoring.

6.9 Operator Dashboard (User Interface) Results

The gateway serves a single-page operator dashboard at `/ui` for interactive testing. Figure 6.7 shows the landing view; Figure 6.8 shows a paraphrase served as a HIT; and Figure 6.9 shows the single clearest proof of the contribution — a look-alike whose similarity (0.944) is *above* the threshold (0.791) yet is rejected with `verify_rejected:number_mismatch`.

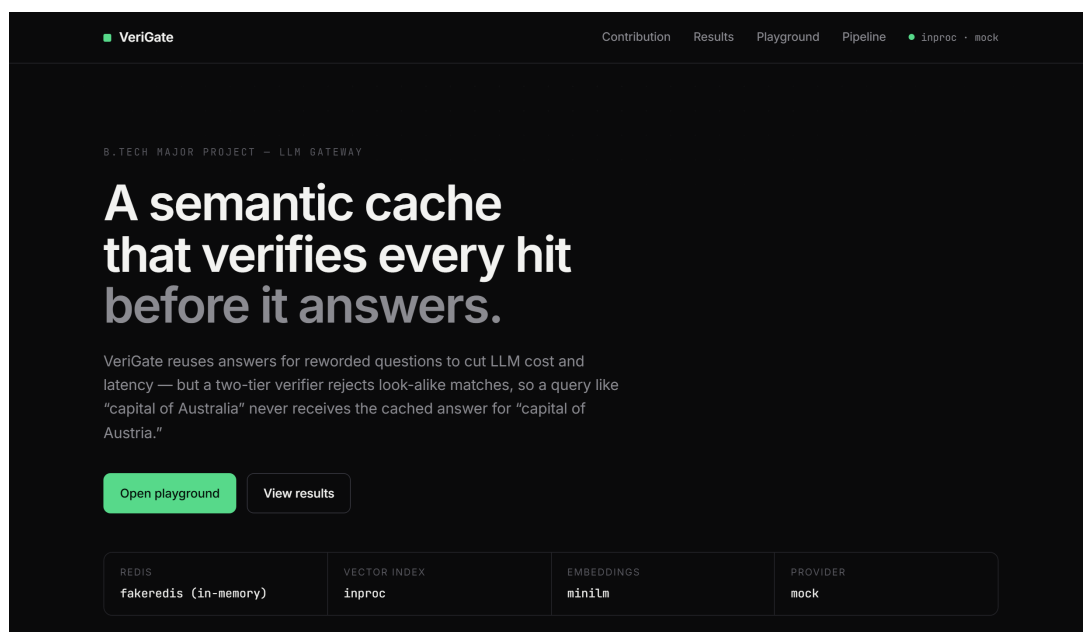


Figure 6.7: VeriGate operator dashboard — landing view. The header reports the live configuration (in-process index, mock provider).

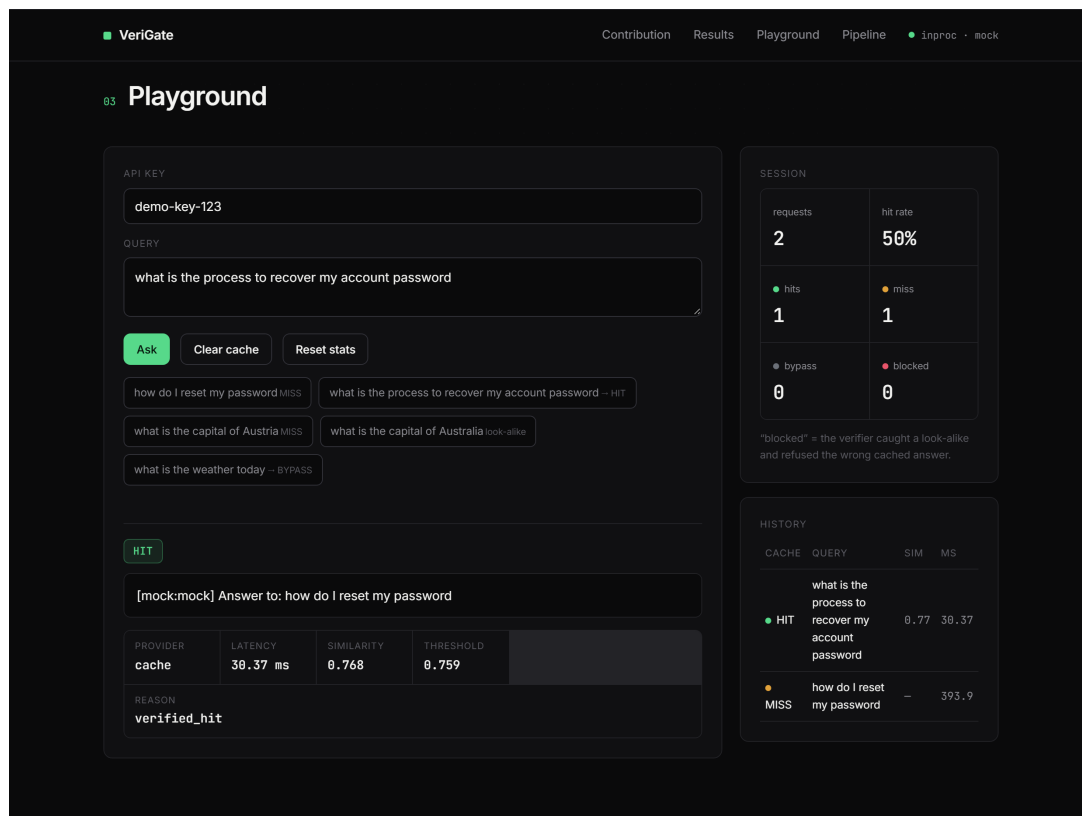


Figure 6.8: A reworded query (“what is the process to recover my account password”) served as a HIT from a prior “how do I reset my password” — matched by meaning, in ~30 ms.

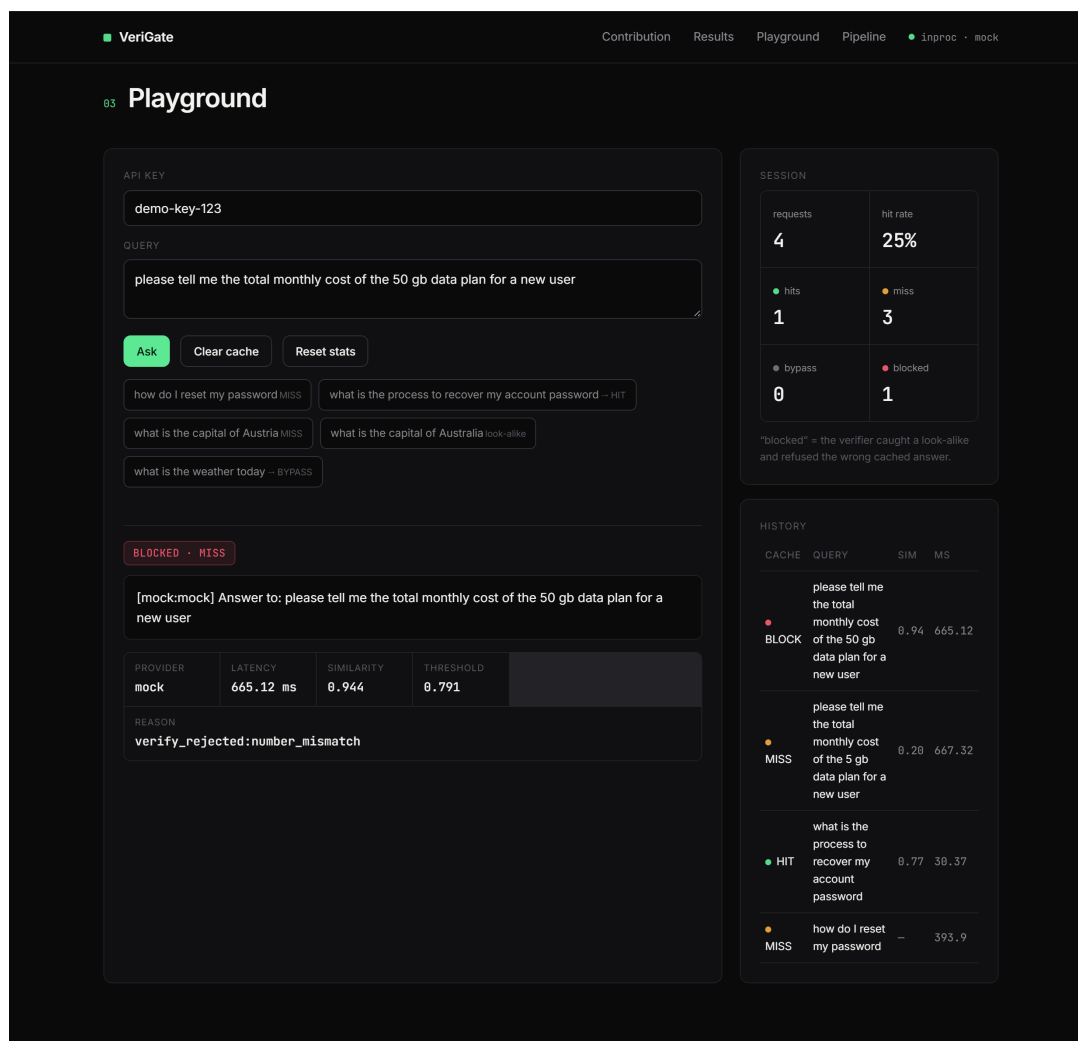


Figure 6.9: The self-verifier in action. The “50 gb” query is similar (0.944) to a cached “5 gb” query — *above* the adaptive threshold (0.791) — yet is **BLOCKED** with reason `verify_rejected:number_mismatch`. This is precisely the false hit a fixed-threshold cache would serve, and precisely what VeriGate prevents.

6.10 Summary of Results

VeriGate reduces the false-hit rate from 46.7% to 0.0% (100% precision) at a structural-verification cost of ~ 0.03 ms; a vectorised index brings the cache-hit path to ~ 10 ms ($\sim 78\times$ faster than an LLM call) and keeps lookup sub-millisecond as the cache grows; the cost-aware cascade cuts LLM spend $\sim 47\%$ with no quality loss on hard queries; and the system is verified secure (tenant isolation), reliable (failover, breaker), observable (Grafana), and live (real Gemini). Collectively these results substantiate every objective of Section 1.4.

Chapter 7

Conclusion

7.1 Conclusion

The literature survey of Chapter 2 identified a clear gap: semantic caches for LLMs deliver real cost and latency savings but trust a single fixed similarity threshold and never verify a candidate hit, making them unsafe in precisely the look-alike cases — differing by a number, a negation, or a named entity — that matter most. In this project we designed, implemented, and evaluated **VeriGate**, a production-grade LLM Gateway whose core novel contribution is an **adaptive, self-verifying semantic cache** that closes this gap by separating the *threshold* decision from the *equivalence* decision: an adaptive threshold proposes a candidate, and a cheap-first two-tier verifier disposes of false hits before any answer is served.

The central result is decisive. On a controlled benchmark of hard negatives, VeriGate reduces the false-hit rate from the fixed-threshold baseline’s **46.7%** to **0.0%** (100% precision), and the ablation by failure mode shows exactly why: the structural verifier eliminates number-, entity-, and negation-swap collisions, while the adaptive threshold clears related-topic ones. Critically, this safety is essentially free — Tier-1 verification costs only **~0.03 ms** on the hit path. A vectorised index reduces the cache-hit path to **~10 ms** (~78× faster than an 800 ms LLM call) and keeps lookup sub-millisecond as the cache scales, and the analysis of the optional Tier-2 NLI verifier yields a genuine research insight: NLI attains maximal correctness but at a recall cost and is largely *redundant* once the adaptive threshold is enabled, so it is best offered as an opt-in “max-correctness mode” rather than a default.

A second contribution, the **cost-aware model cascade**, complements the cache by cutting LLM spend by **~47%** with no quality loss on hard queries, beating random routing at every operating point. Around these two contributions, VeriGate is engineered as a deployable system: per-tenant cache isolation (closing the defining security risk of a semantic cache, with a regression test guarding it), per-key spend budgets that leave cache hits free, a provider circuit breaker and mock failover, Prometheus/Grafana observability, an interactive operator dashboard, and Docker packaging verified end-to-end. The whole thesis was demonstrated live against real Gemini, where a 23-second paid call became a 22 ms free cache hit — even for a reworded question. More broadly, the project shows that in a caching system for high-stakes outputs, *verifying a match before trusting it* is not overhead but the feature itself.

7.2 Limitations of the Current System

Despite the strong results, several limitations bound the current system and should be stated honestly:

1. **Benchmark scale.** The evaluation uses a compact, hand-curated benchmark that isolates the mechanism. The *relative* improvement over the baseline is the result; the *absolute* hit-rate and false-hit numbers are dataset-dependent and would shift on a large natural query log.
2. **Hand-tuned decision logic.** The adaptive-threshold coefficient, the complexity score, and the volatility rules are transparent but heuristic rather than learned, and were tuned on a small validation set.
3. **Tier-1 verifier coverage.** The structural verifier covers numbers, negation, and named entities; subtler semantic distinctions rely on the more expensive Tier-2 NLI stage, which is off by default.
4. **Unbounded cache growth.** A per-entry TTL exists, but a maximum-size cap with LRU/LFU eviction is not yet implemented, so memory is bounded only by TTL.
5. **Operational surface.** `/metrics` is public in the demo build, and TLS, security headers, and dependency scanning are deployment-time concerns documented but not enforced in-app.

7.3 Future Scope

The modular design admits several extensions, grouped by horizon:

- **Near-term — learned threshold and router.** Replace the heuristic adaptive-threshold coefficient and complexity score with lightweight classifiers trained on a validation split, in the style of learned LLM routing [10], and gate Tier-2 NLI to borderline similarities only with a per-request NLI budget.
- **Near-term — cache eviction.** Add a maximum-size cap with LRU/LFU eviction and memory monitoring to bound growth independently of TTL.
- **Medium-term — large-scale evaluation.** Re-run the ablation on a public paraphrase corpus and a real query log at the million-vector scale, tuning the adaptive weights on a held-out split and reporting sensitivity.
- **Medium-term — distributed scale.** Harden the Redisearch/HNSW backend for multi-replica deployments sharing one cache, and tune HNSW parameters at the target corpus size [8].
- **Long-term — richer verification and safety.** Extend the verifier with unit-aware and temporal reasoning, add optional prompt-injection and output-moderation middleware, and encrypt cached content at rest with PII-aware do-not-cache classification.
- **Long-term — productionisation.** Lock down `/metrics`, integrate a host secret store, add TLS/security headers/CORS and CI dependency scanning, and publish a public deployment as a reference service.

Bibliography

- [1] Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), 2023.
- [2] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. A large annotated corpus for learning natural language inference. In *Proceedings of EMNLP*, pages 632–642, 2015.
- [3] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance, 2023.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.
- [5] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations (ICLR)*, 2021.
- [6] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.
- [7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.
- [8] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [9] Michael T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2nd edition, 2018.
- [10] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data, 2024.
- [11] Prometheus Authors. Prometheus: Monitoring system and time series database. <https://prometheus.io>, 2024. Accessed: 2026-10.
- [12] Sebastián Ramírez. FastAPI: A modern, fast web framework for building APIs with python. <https://fastapi.tiangolo.com>, 2024. Accessed: 2026-10.
- [13] Redis Ltd. Redis stack and RediSearch: Vector similarity search. <https://redis.io/docs/interact/search-and-query/advanced-concepts/vectors/>, 2024. Accessed: 2026-10.

- [14] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019.
- [15] Nils Reimers and Iryna Gurevych. Sentence-transformers: all-MiniLM-L6-v2 model card. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>, 2021. Accessed: 2026-10.
- [16] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.
- [18] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [19] Kaijie Zhu, Jindong Wang, et al. A survey on efficient inference for large language models, 2024.